D.Y.Patil International University, Akurdi, Pune

School of Computer Science Engineering and Applications (SCSEA)

Third Year Engineering (B.Tech)

Deep Neural Network (CSE3101)

Lab Manual

D Y PATIL
INTERNATIONAL
UNIVERSITY
AKURDI PUNE

Vision of the University:

"To Create a vibrant learning environment – fostering innovation and creativity, experiential learning, which is inspired by research, and focuses on regionally, nationally and globally relevant areas."

Mission of the School:

- *To provide a diverse, vibrant and inspirational learning environment.*
- *To establish the university as a leading experiential learning and research oriented center.*
- *To become a responsive university serving the needs of industry and society.*
- *To embed internationalization, employability and value thinking*

Deep Neural Network

Course Objectives:

Understand the Mathematics for Neural Networks.

Understanding Functioning of Neural Networks and Kernel Methods.

Understand the Linear Models for Regression and Classification

Understanding Mixture Models.

Course Outcomes:

On completion of the course, learner will be able to–

CO1: Design various types of Neural Networks.

CO2: Construct and Select Kernels for dataset

CO3: Implement linear models for classification and regression

CO4: Implement Bayesian for different applications and design mixture models.

Program Outcomes:

PO1	**Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
PO2	**Problem analysis**: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
PO3	**Design/development of solutions**: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
PO4	**Conduct investigations of complex problems**: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
PO5	**Modern tool usage**: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
PO6	**The engineer and society**: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7	**Environment and sustainability**: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
PO8	**Ethics**: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
PO9	**Individual and team work**: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
PO10	**Communication**: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
PO11	**Project management and finance**: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
PO12	**Life-long learning**: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Rules and Regulations for Laboratory:

- Students should be regular and punctual to all the Lab practical
- Lab assignments and practical should be submitted within given time.
- Mobile phones are strictly prohibited in the Lab.
- Please shut down the Computers before leaving the Lab.
- Please switch off the fans and lights and keep the chair in proper position before leaving the Lab
- Maintain proper discipline in Lab

D.Y.Patil International University, Akurdi, Pune
School of Computer Science Engineering and Applications
Index

Sr. No.	Name of the Practical	Date of Conduction	Page No.		Sign of Teacher	Remarks*
			From	To		
1.	Implement Polynomial Curve fitting using the polyfit function					
2.	Implement a basic multiple layer perceptron for regression and classification on XOR and XNOR truth table					
3.	Build a Basic neural network from scratch using a high level library like tensorflow or Pytorch. Use appropriate dataset					
4.	Optimize hyperparameters for a neural network model. Implement a hyperparameter optimization strategy and compare the performance with different hyperparameter configurations for both classification and regression task					
5.	Develop a Python function (Multivariate Optimization) to compute the Hessian matrix for a given scalar-valued function of multiple variables.					
6.	Implement Bayesian Neural Network					
7.	Implement Regularization in neural Network					
8.	Implement EM (Expectation Maximization) Algorithm					

***Absent/Attended/Late/Partially Completed/Completed**

CERTIFICATE

This is to certify that Mr. Poorva Naik PRN: 20220802219 of class: Btech CSE and track Data Science has completed practical/term work in the course of Deep Neural Network of Third Year Engineering (B. Tech) within SCSEA, as prescribed by D.Y. Patil International University, Pune during the academic year 2024 - 2025.

Date: **Teaching Assistant** **Faculty I/C** **Director** (SCSEA)

Practical 01

Student Name: Poorva Naik
Date of Experiment:
Date of Submission:
PRN No: 20220802219

Aim:-

(a.) Compile and run an OpenMP program that finds the factorial of a number inputted by the user. Also compile the equivalent serial problem (i.e. the openMP directives are ignored).

(b.) And find the computation time for both the serial as well as the parallel; programming and conclude which requires the lesser computation time.

Description:

The algorithm begins by accepting a user input for the number whose factorial is to be computed. In the **serial version**, it iterates from 1 to the given number, multiplying the result in each iteration. For the **parallel version**, OpenMP directives are used to split the loop into multiple threads, each responsible for calculating a portion of the factorial, and the results are combined using a reduction operation. The computation time for both versions is measured using either `clock()` (serial) or `omp_get_wtime()` (parallel) to track the time taken to complete the factorial calculation. To execute the programs, the serial version is compiled normally, while the parallel version requires OpenMP flags (`-fopenmp`) during compilation to enable multi-threading. The results are then displayed along with the computation times for comparison.

.

Program:

OpenMP Program (Parallel Version)

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>

long long factorial(int n) {
   long long result = 1;

   #pragma omp parallel for reduction(*:result)
   for (int i = 1; i <= n; i++) {
      result *= i;
   }
```

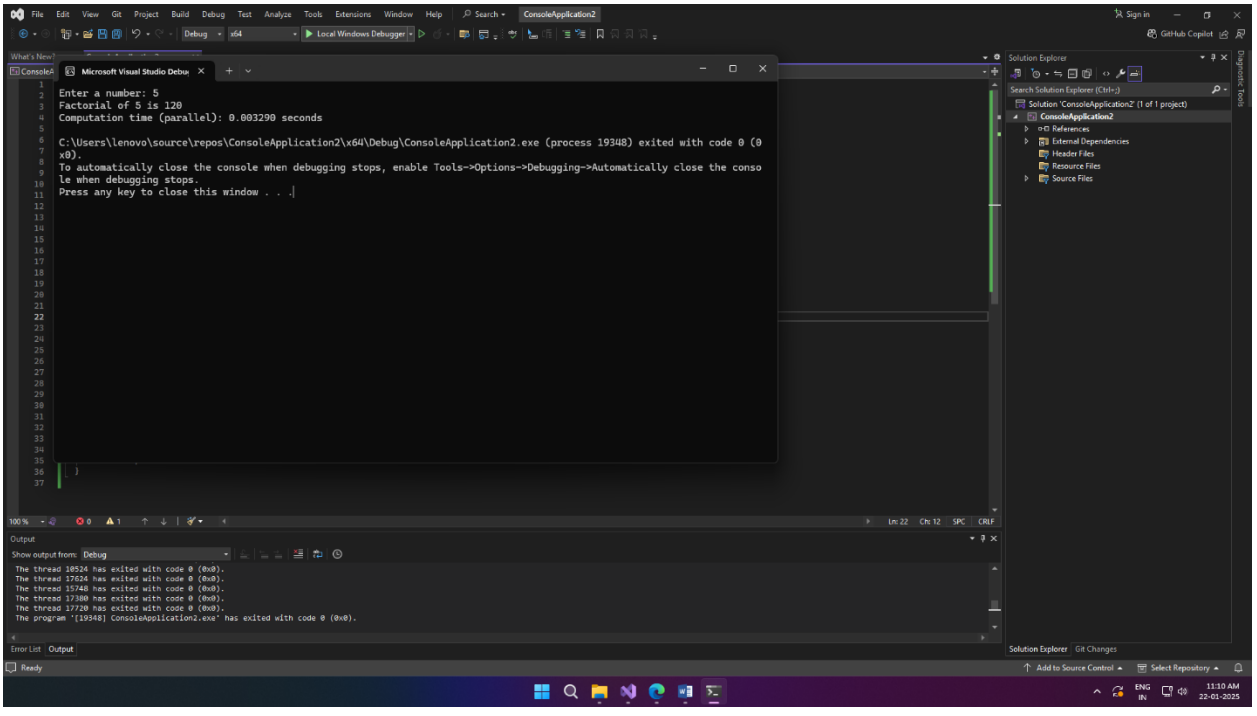
```
    return result;
}

int main() {
    int num;

    // User input
    printf("Enter a number: ");
    scanf("%d", &num);

    // Start timing
    double start_time = omp_get_wtime();

    long long result = factorial(num);

    // End timing
    double end_time = omp_get_wtime();

    printf("Factorial of %d is %lld\n", num, result);
    printf("Computation time (parallel): %f seconds\n", end_time - start_time);

    return 0;
}
```

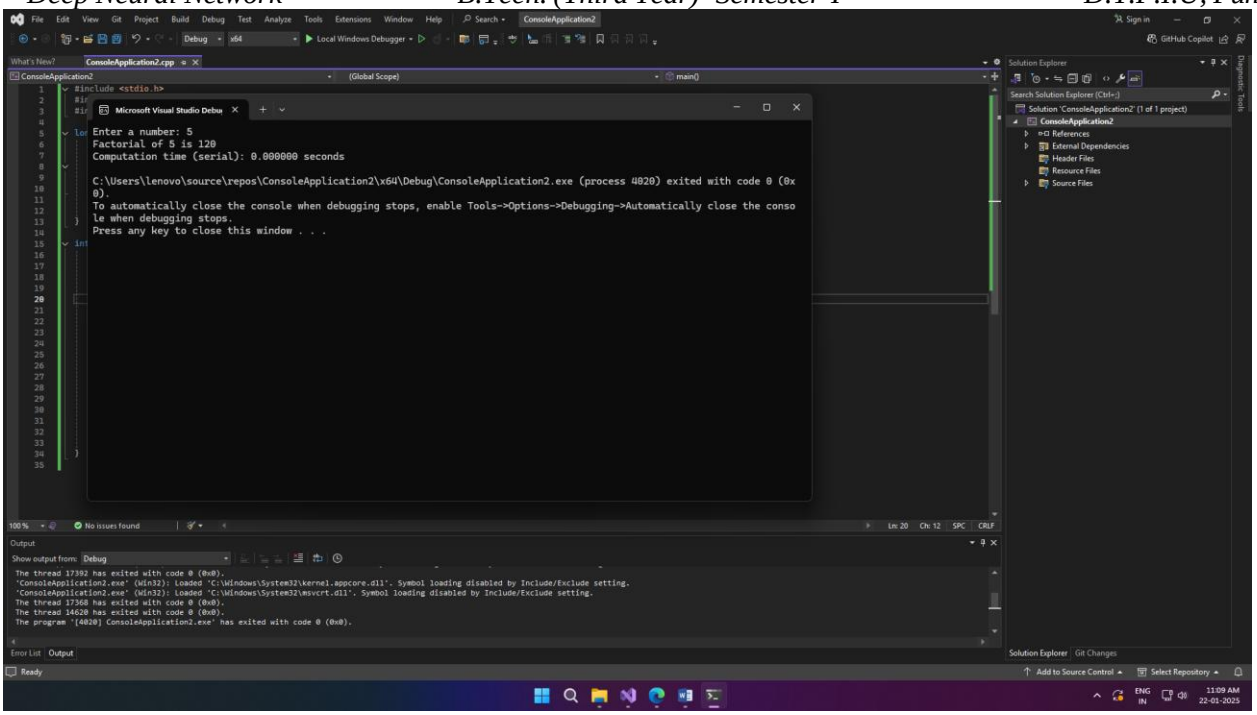
Serial Program

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

long long factorial(int n) {
    long long result = 1;

    for (int i = 1; i <= n; i++) {
        result *= i;
    }

    return result;
}
```

```
int main() {
    int num;

    // User input
    printf("Enter a number: ");
    scanf("%d", &num);

    // Start timing
    clock_t start_time = clock();

    long long result = factorial(num);

    // End timing
    clock_t end_time = clock();

    printf("Factorial of %d is %lld\n", num, result);
    printf("Computation time (serial): %f seconds\n", (double)(end_time - start_time) / CLOCKS_PER_SEC);

    return 0;
}
```

Result:





Conclusion:

The OpenMP parallel version of the factorial computation significantly outperforms the serial version for large inputs by utilizing multiple CPU cores, reducing computation time. However, for small inputs, the overhead of thread management can make the parallel version slower than the serial version. The serial program is simpler and has lower overhead, making it efficient for small tasks but less suitable for large-scale computations. Overall, parallel programming offers scalability and performance improvements for larger problems, while serial execution is more efficient for smaller ones.

Practical 02

Student Name: Poorva Naik
Date of Experiment:
Date of Submission:
PRN No:20220802219

Aim: Implement a multiple layer perceptron for regression and classification on XOR and XNOR truth table

Objectives:

Single Layer Perceptron

1. Define a simple dataset with linearly separable classes.
2. Implement a perceptron with a step activation function.
3. Train the perceptron using the perceptron learning algorithm.
4. Evaluate the model on the dataset.

Multi-Layer Perceptron

1. Define a simple dataset (e.g., XOR problem).
2. Implement a multilayer perceptron architecture.
3. Train the network using backpropagation.
4. Evaluate the model on the dataset.

Software/Tool: Python/Jupyter/Anaconda/Colab

Theory:

A single layer perceptron (SLP) is a feed-forward network based on a threshold transfer function. SLP is the simplest type of artificial neural networks and can only classify linearly separable cases with a binary target (1 , 0). The single layer perceptron does not have a priori knowledge, so the initial weights are assigned randomly. SLP sums all the weighted inputs and if the sum is above the threshold (some predetermined value), SLP is said to be activated (output=1).

Output

$$w_1x_1 + w_2x_2 + \ldots + w_nx_n > \theta \Rightarrow 1$$

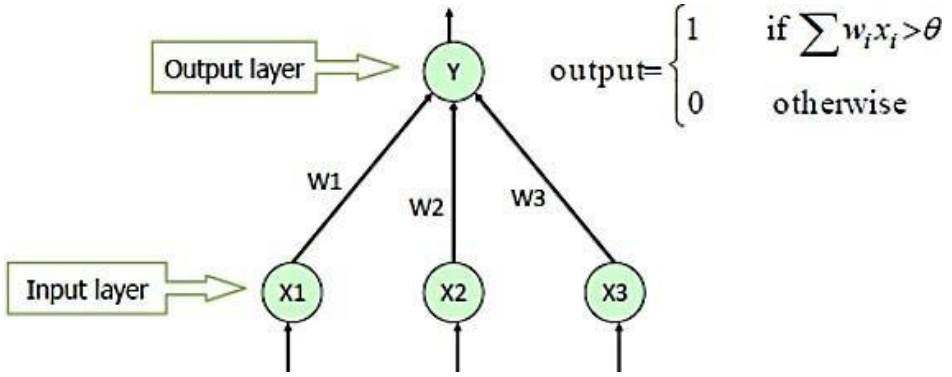
$$w_1x_1 + w_2x_2 + \ldots + w_nx_n \leq \theta \Rightarrow 0$$

$$\Delta w = \eta \times d \times x$$

$d \rightarrow$ Predicted output - Desired output

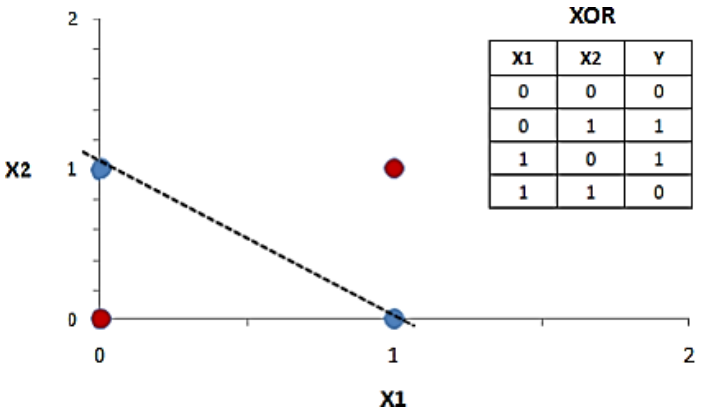
$\eta \rightarrow$ Learning rate, usually less than 1

$x \rightarrow$ Input data



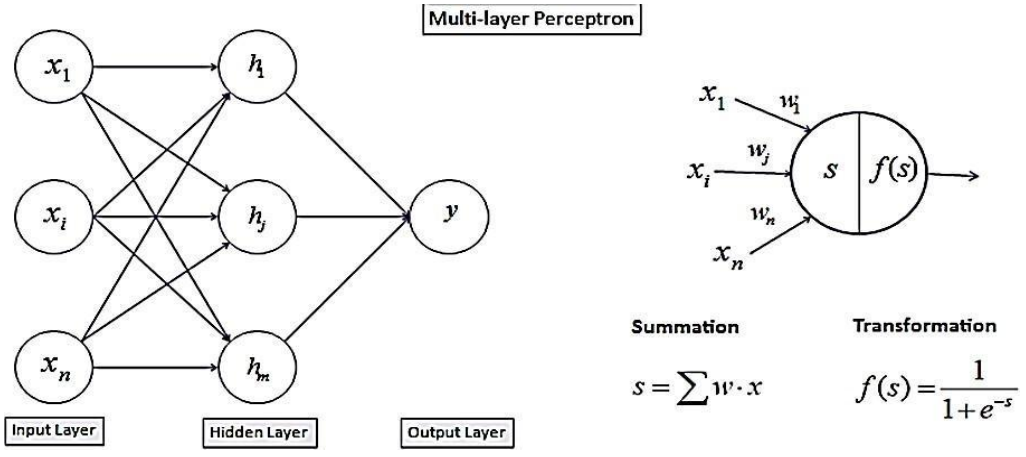
The input values are presented to the perceptron, and if the predicted output is the same as the desired output, then the performance is considered satisfactory and no changes to the weights are made. However, if the output does not match the desired output, then the weights need to be changed to reduce the error.

The most famous example of the inability of perceptron to solve problems with linearly non-separable cases is the XOR problem. However, a multi-layer perceptron using the back propagation algorithm can successfully classify the XOR data. A multi-layer perceptron (MLP) has the same structure of a single layer perceptron with one or more hidden layers.



The back propagation algorithm consists of two phases:

a) the forward phase where the activations are propagated from the input to the output layer, and



b) the backward phase, where the error between the observed actual and the requested nominal value in the output layer is propagated backwards in order to modify the weights and bias values.

1. Error in any ***output*** neuron

$$d_o = y \times (1-y) \times (t-y)$$

2. Error in any ***hidden*** neuron

$$d_i = y_i \times (1-y_i) \times (w_i \times d_o)$$

3. Change the ***weights***

$$\Delta w = \eta \times d \times x$$

Algorithm:

Step 1: Define the Step Function

- This is the activation function.
- If the input is greater than or equal to 0, return 1.
- Else, return 0.

Step 2: Define the Perceptron Function

- Takes inputs, weights, and bias as arguments.
- Calculates the weighted sum: dot product of inputs and weights + bias.
- Applies the step_function to the result to get the final output.

Step 3: Simulate AND Gate

- Inputs: All combinations of two binary inputs.
- Set weights = [1, 1] and bias = -1.5.
- Loop through each input, apply perceptron, and print the result.

Step 4: Simulate OR Gate

- Use the same inputs.
- Set weights = [1, 1] and bias = -0.5.
- Loop through each input, apply perceptron, and print the result.

Step 5: Simulate XOR Gate

- XOR can't be learned by a single perceptron because it's not linearly separable.
- Still, an attempt is made using weights = [1, 1] and bias = -1.

Step 6: Simulate NOR Gate

- Set weights = [-1, -1] and bias = 0.5.
- Loop through each input and print predicted output.

Step 7: Simulate XNOR Gate

- Again, a perceptron alone can't perfectly model XNOR (non-linearly separable).
- Attempt using weights = [1, 1] and bias = -1.

Program code, Results and output

- AND gate

```
import numpy as np

def step_function(x):
    return 1 if x >= 0 else 0

def perceptron(inputs, weights, bias):
    # Calculate the weighted sum
    weighted_sum = np.dot(inputs, weights) + bias
    return step_function(weighted_sum)

# AND Gate: Inputs = [0,0],[0,1],[1,0],[1,1] => Output = [0,0,0,1]
inputs_and = np.array([[0,0], [0,1], [1,0], [1,1]])
weights_and = np.array([1, 1])
bias_and = -1.5

print("AND Gate:")
for i in range(len(inputs_and)):
    predicted_output = perceptron(inputs_and[i], weights_and, bias_and)
    print(f"Input: {inputs_and[i]}, Predicted Output: {predicted_output}")
```

```
AND Gate:
Input: [0 0], Predicted Output: 0
Input: [0 1], Predicted Output: 0
Input: [1 0], Predicted Output: 0
Input: [1 1], Predicted Output: 1
```

- OR gate

```
[ ]
    inputs_and = np.array([[0,0],[0,1],[1,0],[1,1]])
    #outputs_and = np.array([0,0,0,1])
    weights_and = np.array([1,1])
    bias_and = -0.5

    print("OR Gate:")
    for i in range(len(inputs_and)):
        predicted_output = perceptron(inputs_and[i],weights_and,bias_and)
        print(f"input: {inputs_and[i]},Predicted_Output: {predicted_output}")
```

```
OR Gate:
input: [0 0],Predicted_Output: 0
input: [0 1],Predicted_Output: 1
input: [1 0],Predicted_Output: 1
input: [1 1],Predicted_Output: 1
```

- XOR gate

```
    inputs_xor = np.array([[0,0],[0,1],[1,0],[1,1]])
    outputs_xor = np.array([0,1,1,1])
    weights_xor = np.array([1,1])
    bias_xor = -1

    print("\nXOR Gate:")
    for i in range(len(inputs_xor)):
        predicted_output = perceptron(inputs_xor[i],weights_xor,bias_xor)
        print(f"input: {inputs_xor[i]},Predicted_Output: {predicted_output}")
```

```
XOR Gate:
input: [0 0],Predicted_Output: 0
input: [0 1],Predicted_Output: 1
input: [1 0],Predicted_Output: 1
input: [1 1],Predicted_Output: 1
```

- NOR gate

```
[6]
    inputs_xor = np.array([[0,0],[0,1],[1,0],[1,1]])
    outputs_xor = np.array([0,1,1,1])
    weights_xor = np.array([-1,-1])
    bias_xor = 0.5

    print("\nNOR Gate:")
    for i in range(len(inputs_xor)):
        predicted_output = perceptron(inputs_xor[i],weights_xor,bias_xor)
        print(f"input: {inputs_xor[i]},Predicted_Output: {predicted_output}")
```

```
NOR Gate:
input: [0 0],Predicted_Output: 1
input: [0 1],Predicted_Output: 0
input: [1 0],Predicted_Output: 0
input: [1 1],Predicted_Output: 0
```

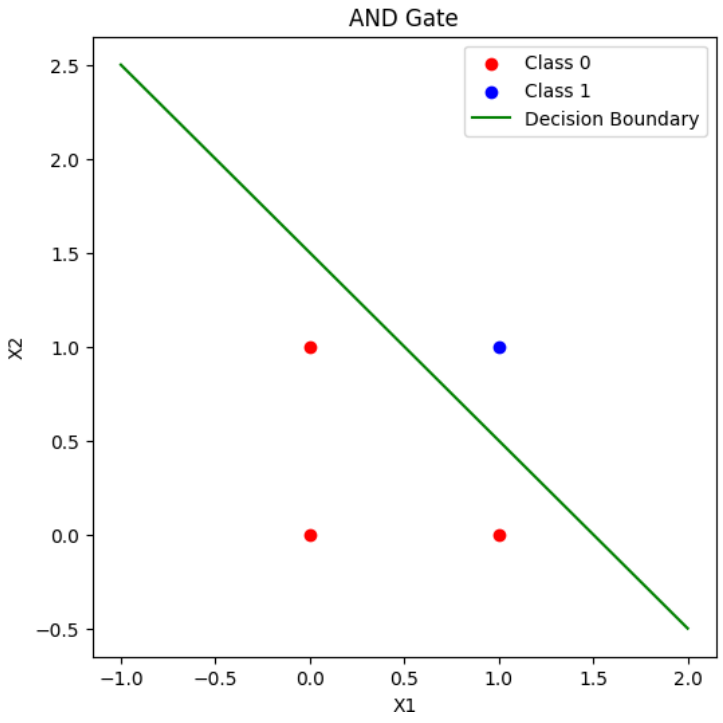
- XNOR gate

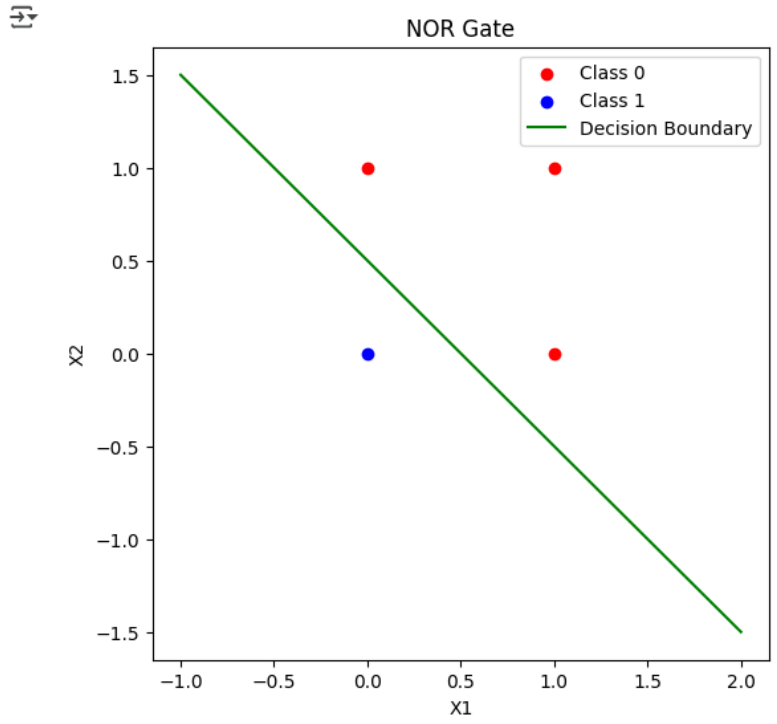
```
inputs_xor = np.array([[0,0],[0,1],[1,0],[1,1]])
outputs_xor = np.array([1,0,0,1])
weights_xor = np.array([1,1])
bias_xor = -1

print("\nXNOR Gate:")
for i in range(len(inputs_xor)):
    predicted_output = perceptron(inputs_xor[i],weights_xor,bias_xor)
    print(f"input: {inputs_xor[i]},Predicted_Output: {predicted_output}")
```

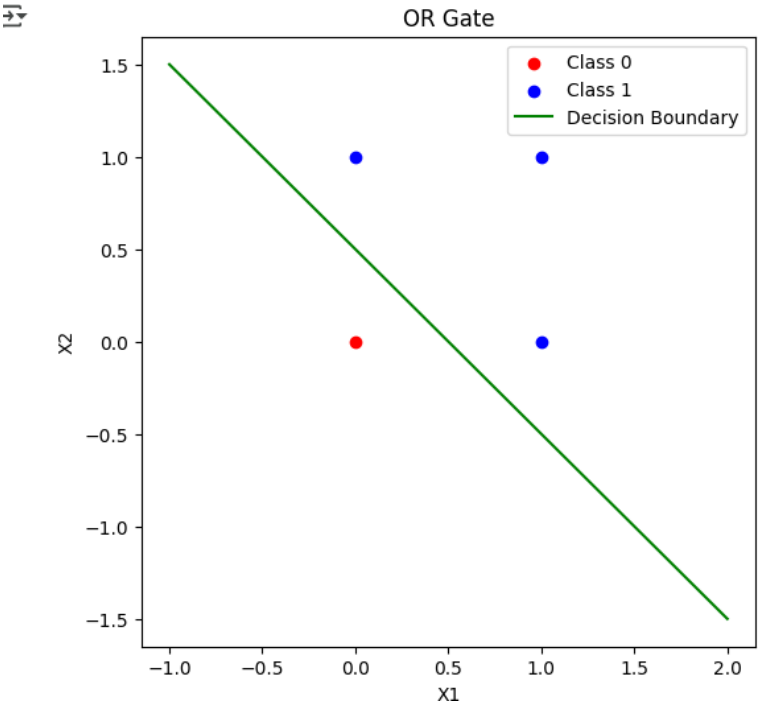
```
XNOR Gate:
input: [0 0],Predicted_Output: 0
input: [0 1],Predicted_Output: 1
input: [1 0],Predicted_Output: 1
input: [1 1],Predicted_Output: 1
```

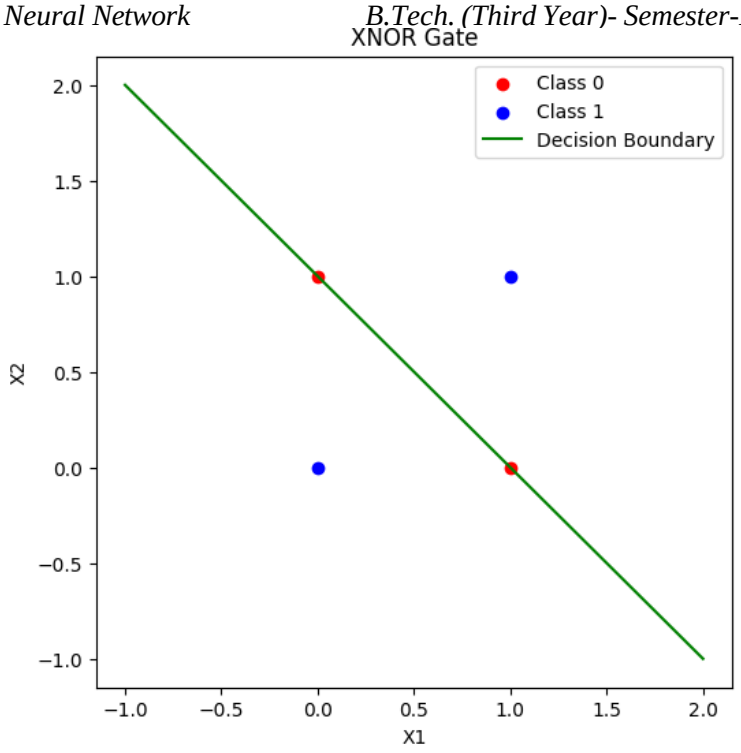
```
import numpy as np
import matplotlib.pyplot as plt

# Perceptron Model for classification
def perceptron(inputs, weights, bias):
    return 1 if np.dot(inputs, weights) + bias >= 0 else 0

# Plotting function for linear separable gates
def plot_gate(inputs, outputs, weights, bias, gate_name):
    plt.figure(figsize=(6, 6))
    plt.title(f'{gate_name} Gate')

    # Plot data points
    plt.scatter(inputs[outputs == 0][:, 0], inputs[outputs == 0][:, 1], color='red', label='Class 0')
    plt.scatter(inputs[outputs == 1][:, 0], inputs[outputs == 1][:, 1], color='blue', label='Class 1')

    # Plot decision boundary (line: w1*x1 + w2*x2 + b = 0)
    x_vals = np.linspace(inputs[:, 0].min() - 1, inputs[:, 0].max() + 1, 100)
    y_vals = (-weights[0] * x_vals - bias) / weights[1]
    plt.plot(x_vals, y_vals, color='green', label='Decision Boundary')

    plt.xlabel('X1')
    plt.ylabel('X2')
    plt.legend()
    plt.show()

# Gate definitions
def and_gate():
    inputs = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
    outputs = np.array([0, 0, 0, 1])
    weights = np.array([1, 1])
    bias = -1.5
    plot_gate(inputs, outputs, weights, bias, "AND")

def or_gate():
    inputs = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
    outputs = np.array([0, 1, 1, 1])
    weights = np.array([1, 1])
    bias = -0.5
    plot_gate(inputs, outputs, weights, bias, "OR")
```

```
    plt.xlabel('X1')
    plt.ylabel('X2')
    plt.legend()
    plt.show()

# Gate definitions
def and_gate():
    inputs = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
    outputs = np.array([0, 0, 0, 1])
    weights = np.array([1, 1])
    bias = -1.5
    plot_gate(inputs, outputs, weights, bias, "AND")

def or_gate():
    inputs = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
    outputs = np.array([0, 1, 1, 1])
    weights = np.array([1, 1])
    bias = -0.5
    plot_gate(inputs, outputs, weights, bias, "OR")

def nor_gate():
    inputs = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
    outputs = np.array([1, 0, 0, 0])
    weights = np.array([-1, -1])
    bias = 0.5
    plot_gate(inputs, outputs, weights, bias, "NOR")

def xnor_gate():
    inputs = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
    outputs = np.array([1, 0, 0, 1])  # XNOR outputs
    weights = np.array([1, 1])
    bias = -1
    plot_gate(inputs, outputs, weights, bias, "XNOR")

# Run the gates
and_gate()
or_gate()
nor_gate()
xnor_gate()
```




NOR Gate
Class 0
Class 1
Decision Boundary
X1
X2
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0


OR Gate
Class 0
Class 1
Decision Boundary
X1
X2
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0


XNOR Gate
Class 0
Class 1
Decision Boundary
X1
X2
2.0
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0

Conclusion:

Practical 03

Student Name: Poorva Naik
Date of Experiment:
Date of Submission:
PRN No:20220802219

Aim: Build a Basic neural network from scratch using a high level library like tensorflow or Pytorch. Use appropriate dataset

Objectives:

1. select two datasets for prediction and classification model
2. implement prediction model using Relu and linear activation functions
3. implement classification model using Relu and Sigmoid activation function

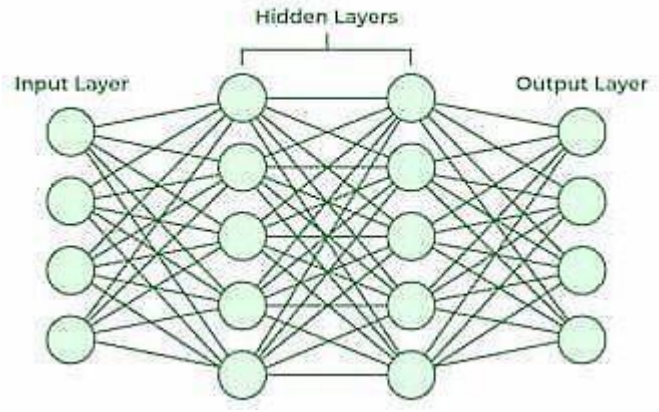
Software/Tool: Python/Jupyter/Anaconda/Colab

Dataset link :
link

Theory:

Neural Networks are computational models that mimic the complex functions of the human brain. The neural networks consist of interconnected nodes or neurons that process and learn from data, enabling tasks such as pattern recognition and decision making in machine learning. **Working of a Neural Network**

Neural networks are complex systems that mimic some features of the functioning of the human brain. It is composed of an input layer, one or more hidden layers, and an output layer made up of layers of artificial neurons that are coupled. The two stages of the basic process are called back propagation and forward propagation.



Forward Propagation

- **Input Layer:** Each feature in the input layer is represented by a node on the network, which receives input data.
- **Weights and Connections:** The weight of each neuronal connection indicates how strong the connection is. Throughout training, these weights are changed.

- **Hidden Layers:** Each hidden layer neuron processes inputs by multiplying them by weights, adding them up, and then passing them through an activation function. By doing this, non- linearity is introduced, enabling the network to recognize intricate patterns.
- **Output:** The final result is produced by repeating the process until the output layer is reached.

Backpropagation

Loss Calculation: The network's output is evaluated against the real goal values, and a loss function is used to compute the difference. For a regression problem, the Mean Squared Error (MSE) is commonly used as the cost function.

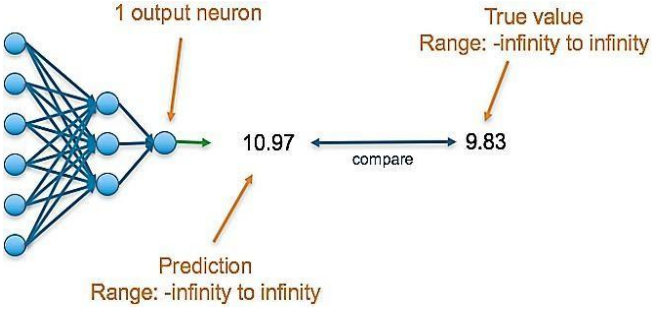
Loss Function:

- **Gradient Descent:** Gradient descent is then used by the network to reduce the loss. To lower the inaccuracy, weights are changed based on the derivative of the loss with respect to each weight.
- **Adjusting weights:** The weights are adjusted at each connection by applying this iterative process, or backpropagation, backward across the network.
- **Training:** During training with different data samples, the entire process of forward propagation, loss calculation, and backpropagation is done iteratively, enabling the network to adapt and learn patterns from the data.
- **Activation Functions:** Model non-linearity is introduced by activation functions like the rectified linear unit (ReLU) or sigmoid. Their decision on whether to "fire" a neuron is based on the whole weighted input.

Predicting a numerical value

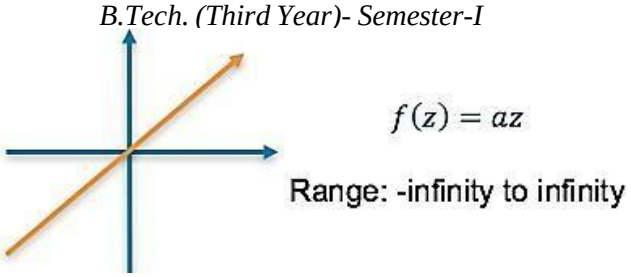
E.g. predicting the price of a product

The final layer of the neural network will have one neuron and the value it returns is a continuous numerical value. To understand the accuracy of the prediction, it is compared with the true value which is also a continuous number.



Final Activation Function

Linear — This results in a numerical value which we require



Or **ReLU** — this results in a numerical value greater than 0

Loss Function

Mean squared error (MSE) — This finds the average squared difference between the predicted value and the true value

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

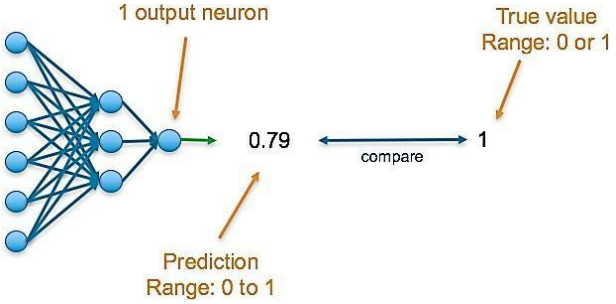
Where $\hat{y}$ is the predicted value and y is the true value

Categorical: Predicting a binary outcome

E.g. predicting a transaction is fraud or not

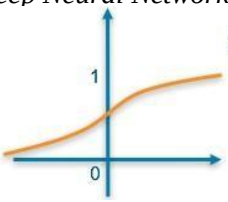
The final layer of the neural network will have one neuron and will return a value between 0 and 1, which can be inferred as a probably.

To understand the accuracy of the prediction, it is compared with the true value. If the data is that class, the true value is a 1, else it is a 0.



Final Activation Function

Sigmoid — This results in a value between 0 and 1 which we can infer to be how confident the model is of the example being in the class



$$sigmoid(z) = \frac{1}{1+e^{-z}}$$

Range: 0 to 1

Loss Function

Binary Cross Entropy — Cross entropy quantifies the difference between two probability distribution. Our model predicts a model distribution of {p, 1-p} as we have a binary distribution. We use binary cross-entropy to compare this with the true distribution {y, 1-y}

$$\text{Binary cross entropy} = -(y\log(\hat{y}) + (1-y)\log(1-\hat{y}))$$

Where $\hat{y}$ is the predicted value and y is the true value

Algorithm

Steps:

1. **Start**
2. **Import Required Libraries**
   - Use tensorflow, keras, matplotlib, and numpy for building and visualizing the model.
3. **Load the MNIST Dataset**
   - Use tf.keras.datasets.mnist.load_data() to load training and testing data.
4. **Normalize the Data**
   - Scale pixel values of the images to range between 0 and 1 by dividing by 255.
5. **Define the Neural Network Architecture**
   - Use Sequential() model:
     - Flatten layer to convert 2D image to 1D array.
     - Dense layer with 128 neurons and ReLU activation.
     - Output Dense layer with 10 neurons (for digits 0–9) using softmax activation.
6. **Compile the Model**
   - Optimizer: adam
   - Loss function: sparse_categorical_crossentropy
   - Metric: accuracy
7. **Train the Model**
   - Use model.fit() with training data and train for a fixed number of epochs (e.g., 5).
8. **Evaluate the Model**
   - Use model.evaluate() with test data to get accuracy and loss.
9. **Visualize Predictions**

- Display test images using matplotlib.
- Use model.predict() to find the predicted label.
- Compare actual label and predicted label

Explanation of the Algorithm for Handwritten Digit Classification

This algorithm uses a neural network to recognize digits from the MNIST dataset. Here's how each part works:

1. **Dataset**
   The MNIST dataset contains grayscale images of handwritten digits. Each image is 28x28 pixels, and the dataset includes labels (0 to 9).
2. **Preprocessing**
   - Input images are normalized to ensure faster convergence during training.
3. **Neural Network Design**
   - The model flattens each image into a 784-element array.
   - A hidden dense layer with 128 neurons captures patterns in the data.
   - The output layer has 10 neurons to classify each digit.
4. **Training and Evaluation**
   - The model learns by adjusting weights based on the error between predicted and actual labels.
   - Accuracy on the test set shows how well the model generalizes.
5. **Prediction and Visualization**
   - Individual images are passed through the trained model.
   - The predicted label is compared to the actual label, and results are visualized using matplotlib.

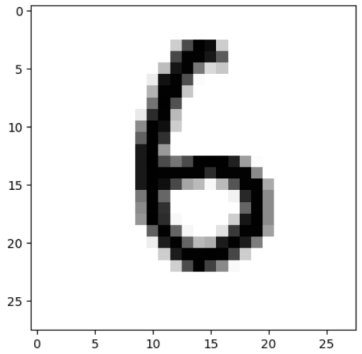
Program code, Results and output

```
[ ] import tensorflow as tf
    from tensorflow.keras import layers, models
    import matplotlib.pyplot as plt

[ ] (x_train,y_train),(x_test,y_test) = tf.keras.datasets.mnist.load_data()

[ ] x_train=x_train/255.0
    x_test=x_test/255.0

[ ] model=models.Sequential([
        layers.Flatten(input_shape=(28,28)),
        layers.Dense(128,activation='relu'),
        layers.Dense(10,activation='softmax')
    ])

/usr/local/lib/python3.11/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential model
  super().__init__(**kwargs)

[ ] model.compile(optimizer='adam',
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])

    model.fit(x_train,y_train,epochs=5)

Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 12s 6ms/step - accuracy: 0.8750 - loss: 0.4381
Epoch 2/5
```

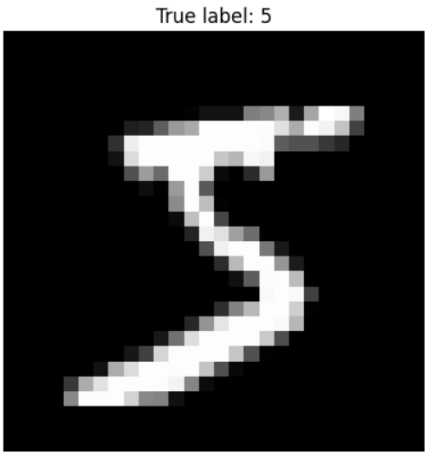
```
[ ] test_loss,test_acc=model.evaluate(x_test,y_test)
    print(f"test accuracy: {test_acc}")
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9720 - loss: 0.0915
test accuracy: 0.9764000177383423
```

```
plt.imshow(x_test[21],cmap=plt.cm.binary)
plt.show()
```



```
[ ] index = 0  # For example, the first image in the dataset

    # Plot the image
    plt.imshow(x_train[index], cmap='gray')  # Use cmap='gray' for grayscale images
    plt.title(f"True label: {y_train[index]}")  # Display the true label as the title
    plt.axis('off')  # Turn off axis labels
    plt.show()
```




```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Reshape
from tensorflow.keras.optimizers import Adam

# Define the model architecture
model = Sequential([
    Dense(8, input_dim=4, activation='relu'),  # First hidden layer
    Dense(4, activation='relu'),               # Second hidden layer
    Dense(4, activation='linear'),             # Output layer (linear activation for regression or binary classification)
    Reshape((2, 2, 1))                         # Reshaping the output to (2, 2, 1)
])

# Compile the model (using MSE loss for regression)
model.compile(optimizer=Adam(learning_rate=0.001), loss='mean_squared_error')

# Print model summary
model.summary()

# Model is now ready for training: model.fit(X_train, y_train)
```

Model: "sequential_3"

Layer (type)	Output Shape	Param #
dense_9 (Dense)	(None, 8)	40
dense_10 (Dense)	(None, 4)	36
dense_11 (Dense)	(None, 4)	20
reshape (Reshape)	(None, 2, 2, 1)	0

Total params: 96 (384.00 B)
Trainable params: 96 (384.00 B)
Non-trainable params: 0 (0.00 B)

```python
import numpy as np

# Activation functions and their derivatives
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)  # Derivative of sigmoid

# Function to initialize weights and biases
def initialize_parameters(manual=False):
    if manual:
        W1 = np.array([[0.5, 0.2], [0.3, 0.7]])  # Shape (2,2)
        b1 = np.array([[0.1], [0.2]])  # Shape (2,1)

        W2 = np.array([[0.6, 0.9], [0.4, 0.8]])  # Shape (2,2)
        b2 = np.array([[0.3], [0.5]])  # Shape (2,1)

        W3 = np.array([[0.7, 0.5]])  # Shape (1,2)
        b3 = np.array([[0.2]])  # Shape (1,1)
    else:
        np.random.seed(42)
        W1 = np.random.randn(2, 2) * 0.01
        b1 = np.zeros((2, 1))
        W2 = np.random.randn(2, 2) * 0.01
        b2 = np.zeros((2, 1))
        W3 = np.random.randn(1, 2) * 0.01
        b3 = np.zeros((1, 1))

    return W1, b1, W2, b2, W3, b3

# Training Data (XOR problem)
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]]).T  # Shape (2,4)

# Input your Y_true values manually
Y_true = np.array([[0, 1, 1, 0]])  # Shape (1,4)

# Choose manual or random initialization
manual_weights = True  # Change to False for random initialization
W1, b1, W2, b2, W3, b3 = initialize_parameters(manual=manual_weights)
```

```python
manual_weights = True  # Change to False for random initialization
W1, b1, W2, b2, W3, b3 = initialize_parameters(manual=manual_weights)

# Store old weights before training
old_W1, old_b1 = W1.copy(), b1.copy()
old_W2, old_b2 = W2.copy(), b2.copy()
old_W3, old_b3 = W3.copy(), b3.copy()

# Training loop
learning_rate = 0.1
epochs = 10000
for epoch in range(epochs):
    # Forward propagation
    Z1 = np.dot(W1, X) + b1
    A1 = sigmoid(Z1)

    Z2 = np.dot(W2, A1) + b2
    A2 = sigmoid(Z2)

    Z3 = np.dot(W3, A2) + b3
    A3 = sigmoid(Z3)  # Final output

    # Compute loss (Binary Cross-Entropy)
    loss = -np.mean(Y_true * np.log(A3 + 1e-8) + (1 - Y_true) * np.log(1 - A3 + 1e-8))

    # Compute error
    error = Y_true - A3

    # Backpropagation
    dZ3 = A3 - Y_true  # Output layer error
    dW3 = (1 / X.shape[1]) * np.dot(dZ3, A2.T)
    db3 = np.mean(dZ3, axis=1, keepdims=True)

    dZ2 = np.dot(W3.T, dZ3) * sigmoid_derivative(A2)
    dW2 = (1 / X.shape[1]) * np.dot(dZ2, A1.T)
    db2 = np.mean(dZ2, axis=1, keepdims=True)

    dZ1 = np.dot(W2.T, dZ2) * sigmoid_derivative(A1)
    dW1 = (1 / X.shape[1]) * np.dot(dZ1, X.T)
    db1 = np.mean(dZ1, axis=1, keepdims=True)
```

```
        print(f"Epoch {epoch}, Loss: {loss:.4f}, Error: {np.mean(error):.4f}")

# Display old and new weights
print("\n===== OLD WEIGHTS AND BIASES =====")
print("W1 (before training):\n", old_W1)
print("b1 (before training):\n", old_b1)
print("W2 (before training):\n", old_W2)
print("b2 (before training):\n", old_b2)
print("W3 (before training):\n", old_W3)
print("b3 (before training):\n", old_b3)

print("\n===== NEW WEIGHTS AND BIASES =====")
print("W1 (after training):\n", W1)
print("b1 (after training):\n", b1)
print("W2 (after training):\n", W2)
print("b2 (after training):\n", b2)
print("W3 (after training):\n", W3)
print("b3 (after training):\n", b3)

# Prediction function
def predict(X):
    Z1 = np.dot(W1, X) + b1
    A1 = sigmoid(Z1)

    Z2 = np.dot(W2, A1) + b2
    A2 = sigmoid(Z2)

    Z3 = np.dot(W3, A2) + b3
    A3 = sigmoid(Z3)

    return (A3 > 0.5).astype(int), A3  # Return both predictions and raw output

# Testing the trained network
predictions, raw_output = predict(X)
error_after_training = Y_true - raw_output

print("\n===== FINAL PREDICTIONS =====")
print(predictions)

print("\n===== ERROR AFTER TRAINING =====")
print(error_after_training)
```

```
Epoch 0, Loss: 0.8458, Error: -0.2569
Epoch 1000, Loss: 0.6928, Error: -0.0004
Epoch 2000, Loss: 0.6927, Error: -0.0006
Epoch 3000, Loss: 0.6925, Error: -0.0007
Epoch 4000, Loss: 0.6921, Error: -0.0010
Epoch 5000, Loss: 0.6913, Error: -0.0014
Epoch 6000, Loss: 0.6895, Error: -0.0021
Epoch 7000, Loss: 0.6850, Error: -0.0033
Epoch 8000, Loss: 0.6681, Error: -0.0061
Epoch 9000, Loss: 0.5988, Error: -0.0104

===== OLD WEIGHTS AND BIASES =====
W1 (before training):
 [[0.5 0.2]
 [0.3 0.7]]
b1 (before training):
 [[0.1]
 [0.2]]
W2 (before training):
 [[0.6 0.9]
 [0.4 0.8]]
b2 (before training):
 [[0.3]
 [0.5]]
W3 (before training):
 [[0.7 0.5]]
b3 (before training):
 [[0.2]]

===== NEW WEIGHTS AND BIASES =====
W1 (after training):
 [[1.13217597 0.99311616]
 [4.05177701 4.06620092]]
b1 (after training):
 [[ 0.05369569]
 [-1.18817762]]
W2 (after training):
 [[1.74595170e-04 4.25864536e+00]
 [2.65014538e-02 1.59556305e+00]]
b2 (after training):
 [[-2.00243516]
 [-0.32921731]]
W3 (after training):
 [[3.77205409 0.68991188]]
b3 (after training):
 [[-3.33609318]]

===== FINAL PREDICTIONS =====
[[0 1 1 1]]

===== ERROR AFTER TRAINING =====
[[-0.1221259   0.37092867  0.37122407 -0.64972774]]
```

```
[ ] import torch
    import torch.nn as nn
    import torch.optim as optim
    import torchvision
    import torchvision.transforms as transforms
    from torch.utils.data import DataLoader

    # --------------------------
    # 1. Load and Normalize the MNIST Dataset
    # --------------------------
    transform = transforms.Compose([
        transforms.ToTensor(),  # Convert PIL image to tensor
        transforms.Normalize((0.5,), (0.5,))  # Normalize to [-1, 1]
    ])

    train_dataset = torchvision.datasets.MNIST(root='./data', train=True, download=True, transform=transform)
    test_dataset = torchvision.datasets.MNIST(root='./data', train=False, download=True, transform=transform)

    train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
    test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)

    # --------------------------
    # 2. Define a Simple Feedforward Neural Network
    # --------------------------
    class SimpleNN(nn.Module):
        def __init__(self):
            super(SimpleNN, self).__init__()
            self.fc1 = nn.Linear(28*28, 128)
            self.fc2 = nn.Linear(128, 64)
            self.fc3 = nn.Linear(64, 10)
            self.relu = nn.ReLU()

        def forward(self, x):
            x = x.view(x.size(0), -1)  # Flatten the image
            x = self.relu(self.fc1(x))
            x = self.relu(self.fc2(x))
            x = self.fc3(x)  # No activation before CrossEntropyLoss
            return x
```

```
    # --------------------------
    # 3. Initialize Model, Loss, and Optimizer
    # --------------------------
    model = SimpleNN()
    loss_fn = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001)

    # --------------------------
    # 4. Training Loop
    # --------------------------
    epochs = 5

    for epoch in range(epochs):
        model.train()
        total_loss = 0

        for images, labels in train_loader:
            optimizer.zero_grad()
            outputs = model(images)
            loss = loss_fn(outputs, labels)
            loss.backward()
            optimizer.step()
            total_loss += loss.item()

        print(f"Epoch {epoch+1}/{epochs}, Loss: {total_loss/len(train_loader):.4f}")

    # --------------------------
    # 5. Evaluate on Test Set
    # --------------------------
    model.eval()
    correct = 0
    total = 0

    with torch.no_grad():
        for images, labels in test_loader:
            outputs = model(images)
            _, predicted = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    accuracy = 100 * correct / total
```

```
accuracy = 100 * correct / total
print(f"Test Accuracy: {accuracy:.2f}%")
```

```
100%|████████| 9.91M/9.91M [00:01<00:00, 5.42MB/s]
100%|████████| 28.9k/28.9k [00:00<00:00, 160kB/s]
100%|████████| 1.65M/1.65M [00:01<00:00, 1.51MB/s]
100%|████████| 4.54k/4.54k [00:00<00:00, 6.06MB/s]
Epoch 1/5, Loss: 0.3931
Epoch 2/5, Loss: 0.1900
Epoch 3/5, Loss: 0.1383
Epoch 4/5, Loss: 0.1128
Epoch 5/5, Loss: 0.0957
Test Accuracy: 96.52%
```

Conclusion:

Practical 04

Student Name: Poorva Naik
Date of Experiment:
Date of Submission:
PRN No:20220802219

Aim: Optimize hyperparameters for a neural network model. Implement a hyperparameter optimization strategy and compare the performance with different hyperparameter configurations for both classification and regression task.

Objectives:

1. Choose a dataset and neural network architecture.
2. Define a range of hyperparameters to tune.
3. Implement a hyperparameter optimization strategy (e.g. grid search, random search).
4. Compare the performance with different hyperparameter configurations for both classification and regression tasks.

Software/Tool: Python/Jupyter/Anaconda/Colab

Dataset link : link

Theory:

Hyper parameters are the variables which determines the network structure (Eg: Number of Hidden Units) and the variables which determine how the network is trained (Eg: Learning Rate). Hyper parameters are set before training (before optimizing the weights and bias).

Hyper parameter related to Neural Networks:

1) Number of Hidden layers

- Many hidden units within a layer with regularization techniques can increase accuracy. Smaller number of units may cause underfitting.

2) Dropout

- It is regularization technique to avoid overfitting thus increasing the generalizing power.
- Generally, use a small dropout value of 20%-50% of neurons with 20% providing a good starting point.
- A probability too low has minimal effect and value too high results in under-learning by the network.
- You are likely to get better performance when dropout is used on a larger network, giving the model more of an opportunity to learn independent representations. Learning The Hyperparameters

3) Network Weight Initialization

- Ideally, it may be better to use different weight initialization schemes according to the activation function used on each layer.

4) Activation Function

- Activation functions are used to introduce nonlinearity to models, which allows deep learning models to learn nonlinear prediction boundaries.
- Generally, the rectifier activation function (ReLU) is the most popular.
- Sigmoid is used in the output layer while making binary predictions.
- Softmax is used in the output layer while making multi-class predictions.

Methods used to find out hyper parameters

1) Manual Search

2) Grid Search

3) Random Search

4) Bayesian Optimization

Algorithm

Steps:

Start

1. Import necessary libraries: pandas, numpy, tensorflow, kerastuner, sklearn.
2. Load the cleaned mushroom dataset (mushroom_cleaned.csv).
3. Split the dataset into features (X) and target (Y).
4. Divide the dataset into training and testing sets using an 80-20 ratio.
5. Define a model builder function build_model(hp):
    - Accepts a HyperParameters object.
    - Adds an input layer matching the number of features.
    - Adds 1 to 5 hidden layers (tunable).
    - Each hidden layer has 16–256 neurons (tunable, step size 16).
    - Uses ReLU activation for hidden layers.
    - Adds an output layer with sigmoid activation (binary classification).
    - Compiles the model using Adam optimizer with a tunable learning rate (0.01, 0.001, 0.0001).
6. Build and train an **initial model** without tuning for 10 epochs.
7. Evaluate and print the accuracy of the initial model.
8. Initialize the **RandomSearch tuner** from Keras Tuner:
    - Set objective to maximize validation accuracy.
    - Set number of trials to 5.
    - Define a log directory and project name.
9. Run the tuner using the training data and validate on the test set.
10. Retrieve the best hyperparameters found.

11. Build a new model using these best hyperparameters.
12. Train the optimized model on the same dataset for 10 epochs.
13. Evaluate and print the final accuracy of the tuned model.

Explanation of the Algorithm for Hyperparameter Tuning

This algorithm uses **automated hyperparameter search** to find the best neural network architecture for classifying mushrooms as **edible or poisonous**.

1. **Input**
    - Each row in the dataset represents a mushroom with categorical features like gill color, cap shape, etc.
    - The target is binary: edible (0) or poisonous (1).
2. **Start**
    - Begin by importing the necessary libraries and loading the preprocessed dataset.
3. **Data Preparation**
    - Features and target are separated.
    - Data is split into training and testing sets for fair evaluation.
4. **Model Definition with Tuning Options**
    - Define a flexible model builder function using Keras Tuner's HyperParameters.
    - This lets you explore combinations of layer count, units per layer, and learning rate.
5. **Initial Model Evaluation**
    - Train and evaluate a basic model with default settings to use as a baseline.
6. **Hyperparameter Tuning with Random Search**
    - Try multiple combinations of hyperparameters randomly over several trials.
    - Use validation accuracy as the evaluation metric.
    - Identify the best-performing configuration.
7. **Model Retraining**
    - Build a final model with the best hyperparameters.
    - Train and evaluate it again on the test data.
8. **Comparison and Evaluation**
    - Compare the performance of the tuned model with the initial model.
    - Choose the better model for deployment or further testing.

Program code, Results and output

[9]:
```
import pandas as pd
import numpy as np
import tensorflow as tf
from sklearn.model_selection import train_test_split
from kerastuner.tuners import RandomSearch
from kerastuner.engine.hyperparameters import HyperParameters

# Load the dataset
data = pd.read_csv("D:\College\DNN\DNN Lab Exam Data Sets\Lab4\mushroom_cleaned.csv")

# Separate features and target
X = data.drop(columns=["class"]).values
Y = data["class"].values

# Split the data into train and test sets
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42)

# Define the model builder function
def build_model(hp):
    model = tf.keras.Sequential()
    model.add(tf.keras.layers.Input(shape=X_train.shape[1:]))

    # Tune the number of layers and units in each layer
    for i in range(hp.Int('num_layers', 1, 5)):
        model.add(tf.keras.layers.Dense(units=hp.Int('units_' + str(i), min_value=16, max_value=256, step=16),
                                        activation='relu'))

    model.add(tf.keras.layers.Dense(1, activation='sigmoid'))

    # Tune the learning rate
    hp_learning_rate = hp.Choice('learning_rate', values=[1e-2, 1e-3, 1e-4])

    # Compile the model
    model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=hp_learning_rate),
                  loss='binary_crossentropy',
                  metrics=['accuracy'])

    return model
```

```
    return model

# Build and evaluate the initial model
initial_model = build_model(HyperParameters())
initial_model.fit(X_train, Y_train, epochs=10, validation_data=(X_test, Y_test))
_, initial_accuracy = initial_model.evaluate(X_test, Y_test)

print(f"Initial Model Accuracy: {initial_accuracy}")

# Initialize the RandomSearch tuner
tuner = RandomSearch(
    build_model,
    objective='val_accuracy',
    max_trials=5,
    directory='keras_tuner_logs',
    project_name='mushroom_classification'
)

# Perform the hyperparameter search
tuner.search(X_train, Y_train, epochs=10, validation_data=(X_test, Y_test))

# Get the best hyperparameters
best_hps = tuner.get_best_hyperparameters(num_trials=1)[0]

print("Best Hyperparameters:")
print(best_hps)

# Build and train the model with the best hyperparameters
model = tuner.hypermodel.build(best_hps)
model.fit(X_train, Y_train, epochs=10, validation_data=(X_test, Y_test))

# Evaluate the best model
_, best_model_accuracy = model.evaluate(X_test, Y_test)
print(f"Best Model Accuracy: {best_model_accuracy}")
print(f"Initial Model Accuracy: {initial_accuracy}")
```

```
Trial 5 Complete [00h 00m 19s]
val_accuracy: 0.5457573533058167

Best val_accuracy So Far: 0.6897381544113159
Total elapsed time: 00h 01m 25s
Best Hyperparameters:
<keras_tuner.src.engine.hyperparameters.hyperparameters.HyperParameters object at 0x0000016C04716550>
Epoch 1/10
1351/1351 ━━━━━━━━━━━━━━━━━━━━ 3s 1ms/step - accuracy: 0.5623 - loss: 1.5259 - val_accuracy: 0.5953 - val_loss: 0.6451
Epoch 2/10
1351/1351 ━━━━━━━━━━━━━━━━━━━━ 2s 1ms/step - accuracy: 0.5842 - loss: 0.6586 - val_accuracy: 0.6129 - val_loss: 0.6244
Epoch 3/10
1351/1351 ━━━━━━━━━━━━━━━━━━━━ 2s 1ms/step - accuracy: 0.6018 - loss: 0.6413 - val_accuracy: 0.5856 - val_loss: 0.6493
Epoch 4/10
1351/1351 ━━━━━━━━━━━━━━━━━━━━ 2s 1ms/step - accuracy: 0.6000 - loss: 0.6355 - val_accuracy: 0.5622 - val_loss: 0.6562
Epoch 5/10
1351/1351 ━━━━━━━━━━━━━━━━━━━━ 2s 1ms/step - accuracy: 0.5689 - loss: 0.6421 - val_accuracy: 0.5770 - val_loss: 0.6337
Epoch 6/10
1351/1351 ━━━━━━━━━━━━━━━━━━━━ 2s 1ms/step - accuracy: 0.5793 - loss: 0.6271 - val_accuracy: 0.6115 - val_loss: 0.6165
Epoch 7/10
1351/1351 ━━━━━━━━━━━━━━━━━━━━ 2s 1ms/step - accuracy: 0.5870 - loss: 0.6207 - val_accuracy: 0.5696 - val_loss: 0.6493
Epoch 8/10
1351/1351 ━━━━━━━━━━━━━━━━━━━━ 2s 1ms/step - accuracy: 0.5845 - loss: 0.6254 - val_accuracy: 0.6267 - val_loss: 0.6199
Epoch 9/10
1351/1351 ━━━━━━━━━━━━━━━━━━━━ 2s 1ms/step - accuracy: 0.6111 - loss: 0.6057 - val_accuracy: 0.6290 - val_loss: 0.5895
Epoch 10/10
1351/1351 ━━━━━━━━━━━━━━━━━━━━ 2s 1ms/step - accuracy: 0.6062 - loss: 0.6014 - val_accuracy: 0.6400 - val_loss: 0.5914
338/338 ━━━━━━━━━━━━━━━━━━━━ 0s 731us/step - accuracy: 0.6402 - loss: 0.5938
Best Model Accuracy: 0.6399555802345276
Initial Model Accuracy: 0.6184880137443542
```

```python
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.optimizers import Adam, SGD  # Importing SGD optimizer
from tensorflow.keras.initializers import HeNormal  # For He initialization
from tensorflow.keras import regularizers  # For L1, L2 regularization

def build_model(units, lr, dropout_rate, reg_rate, momentum):
    model = models.Sequential([
        layers.Flatten(input_shape=(28, 28)),
        layers.Dense(units, activation='relu',
                     kernel_initializer=HeNormal(),
                     kernel_regularizer=regularizers.l2(reg_rate)),  # L2 Regularization
        layers.Dropout(dropout_rate),  # Dropout layer
        layers.Dense(10, activation='softmax',
                     kernel_initializer=HeNormal(),
                     kernel_regularizer=regularizers.l2(reg_rate))  # L2 Regularization for output layer
    ])

    # Using SGD with momentum
    optimizer = SGD(learning_rate=lr, momentum=momentum)

    model.compile(optimizer=optimizer,
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])
    return model

# Load the MNIST dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

# Normalize the images to [0, 1]
x_train, x_test = x_train / 255.0, x_test / 255.0

# Hyperparameters to test
hidden_units = [64, 128]
learning_rates = [0.001, 0.01]
dropout_rates = [0.2, 0.5]  # Try dropout rates of 20% and 50%
regularization_rates = [0.01, 0.1]  # L2 regularization strength
momentum_values = [0.9, 0.99]  # Momentum values to test (commonly used values)
```

```python
# Normalize the images to [0, 1]
x_train, x_test = x_train / 255.0, x_test / 255.0

# Hyperparameters to test
hidden_units = [64, 128]
learning_rates = [0.001, 0.01]
dropout_rates = [0.2, 0.5]  # Try dropout rates of 20% and 50%
regularization_rates = [0.01, 0.1]  # L2 regularization strength
momentum_values = [0.9, 0.99]  # Momentum values to test (commonly used values)

best_acc = 0
best_params = None

# Loop through the combinations of hyperparameters
for units in hidden_units:
    for lr in learning_rates:
        for dropout_rate in dropout_rates:
            for reg_rate in regularization_rates:
                for momentum in momentum_values:
                    print(f"Training model with {units} hidden units, learning rate {lr}, "
                          f"dropout rate {dropout_rate}, L2 regularization rate {reg_rate}, and momentum {momentum}")
                    model = build_model(units, lr, dropout_rate, reg_rate, momentum)

                    # Train the model for 3 epochs
                    model.fit(x_train, y_train, epochs=3, verbose=0)

                    # Evaluate the model on the test set
                    test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
                    print(f"Test Accuracy: {test_acc:.4f}\n")

                    # Update the best accuracy and parameters
                    if test_acc > best_acc:
                        best_acc = test_acc
                        best_params = (units, lr, dropout_rate, reg_rate, momentum)

# Print the best hyperparameters and the best accuracy
print(f"Best Hyperparameters: Hidden Units = {best_params[0]}, Learning Rate = {best_params[1]}, "
      f"Dropout Rate = {best_params[2]}, L2 Regularization Rate = {best_params[3]}, Momentum = {best_params[4]}")
print(f"Best Accuracy Achieved: {best_acc:.4f}")
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Training model with 64 hidden units, learning rate 0.001, dropout rate 0.2, L2 regularization rate 0.01, and momentum 0.9
/usr/local/lib/python3.11/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input_shape
  super().__init__(**kwargs)
Test Accuracy: 0.9090

Training model with 64 hidden units, learning rate 0.001, dropout rate 0.2, L2 regularization rate 0.01, and momentum 0.99
Test Accuracy: 0.9122

Training model with 64 hidden units, learning rate 0.001, dropout rate 0.2, L2 regularization rate 0.1, and momentum 0.9
Test Accuracy: 0.7621

Training model with 64 hidden units, learning rate 0.001, dropout rate 0.2, L2 regularization rate 0.1, and momentum 0.99
Test Accuracy: 0.6421

Training model with 64 hidden units, learning rate 0.001, dropout rate 0.5, L2 regularization rate 0.01, and momentum 0.9
Test Accuracy: 0.9044

Training model with 64 hidden units, learning rate 0.001, dropout rate 0.5, L2 regularization rate 0.01, and momentum 0.99
Test Accuracy: 0.8989

Training model with 64 hidden units, learning rate 0.001, dropout rate 0.5, L2 regularization rate 0.1, and momentum 0.9
Test Accuracy: 0.7509

Training model with 64 hidden units, learning rate 0.001, dropout rate 0.5, L2 regularization rate 0.1, and momentum 0.99
Test Accuracy: 0.6714

Training model with 64 hidden units, learning rate 0.01, dropout rate 0.2, L2 regularization rate 0.01, and momentum 0.9
Test Accuracy: 0.9101

Training model with 64 hidden units, learning rate 0.01, dropout rate 0.2, L2 regularization rate 0.01, and momentum 0.99
Test Accuracy: 0.8737

Training model with 64 hidden units, learning rate 0.01, dropout rate 0.2, L2 regularization rate 0.1, and momentum 0.9
Test Accuracy: 0.6974

Training model with 64 hidden units, learning rate 0.01, dropout rate 0.2, L2 regularization rate 0.1, and momentum 0.99
Test Accuracy: 0.6362

Training model with 64 hidden units, learning rate 0.01, dropout rate 0.5, L2 regularization rate 0.01, and momentum 0.9
Test Accuracy: 0.9017

Training model with 64 hidden units, learning rate 0.01, dropout rate 0.5, L2 regularization rate 0.01, and momentum 0.99
```

```
Training model with 64 hidden units, learning rate 0.01, dropout rate 0.5, L2 regularization rate 0.01, and momentum 0.99
Test Accuracy: 0.8192

Training model with 64 hidden units, learning rate 0.01, dropout rate 0.5, L2 regularization rate 0.1, and momentum 0.9
Test Accuracy: 0.6872

Training model with 64 hidden units, learning rate 0.01, dropout rate 0.5, L2 regularization rate 0.1, and momentum 0.99
Test Accuracy: 0.5643

Training model with 128 hidden units, learning rate 0.001, dropout rate 0.2, L2 regularization rate 0.01, and momentum 0.9
Test Accuracy: 0.9095

Training model with 128 hidden units, learning rate 0.001, dropout rate 0.2, L2 regularization rate 0.01, and momentum 0.9
Test Accuracy: 0.9084

Training model with 128 hidden units, learning rate 0.001, dropout rate 0.2, L2 regularization rate 0.1, and momentum 0.9
Test Accuracy: 0.7669

Training model with 128 hidden units, learning rate 0.001, dropout rate 0.2, L2 regularization rate 0.1, and momentum 0.99
Test Accuracy: 0.7191

Training model with 128 hidden units, learning rate 0.001, dropout rate 0.5, L2 regularization rate 0.01, and momentum 0.9
Test Accuracy: 0.9095

Training model with 128 hidden units, learning rate 0.001, dropout rate 0.5, L2 regularization rate 0.01, and momentum 0.9
Test Accuracy: 0.9069

Training model with 128 hidden units, learning rate 0.001, dropout rate 0.5, L2 regularization rate 0.1, and momentum 0.9
Test Accuracy: 0.7470

Training model with 128 hidden units, learning rate 0.001, dropout rate 0.5, L2 regularization rate 0.1, and momentum 0.99
Test Accuracy: 0.6990

Training model with 128 hidden units, learning rate 0.01, dropout rate 0.2, L2 regularization rate 0.01, and momentum 0.9
Test Accuracy: 0.9155

Training model with 128 hidden units, learning rate 0.01, dropout rate 0.2, L2 regularization rate 0.01, and momentum 0.99
Test Accuracy: 0.7998

Training model with 128 hidden units, learning rate 0.01, dropout rate 0.2, L2 regularization rate 0.1, and momentum 0.9
Test Accuracy: 0.7268

Training model with 128 hidden units, learning rate 0.01, dropout rate 0.2, L2 regularization rate 0.1, and momentum 0.99
```

```
[ ] #Grid Search for MNIST Classification
```

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader, random_split
import itertools

# Define hyperparameter grid
param_grid = {
    'lr': [0.001, 0.01],
    'hidden_units': [64, 128],
    'batch_size': [64, 128]
}

combinations = list(itertools.product(*param_grid.values()))

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])
train_dataset = torchvision.datasets.MNIST(root='./data', train=True, download=True, transform=transform)
train_data, val_data = random_split(train_dataset, [50000, 10000])
test_dataset = torchvision.datasets.MNIST(root='./data', train=False, transform=transform)

results = []

for lr, hidden_units, batch_size in combinations:
    train_loader = DataLoader(train_data, batch_size=batch_size, shuffle=True)
    val_loader = DataLoader(val_data, batch_size=batch_size, shuffle=False)

    # Define model
    class Net(nn.Module):
        def __init__(self):
            super().__init__()
            self.model = nn.Sequential(
                nn.Flatten(),
                nn.Linear(28*28, hidden_units)
```

```
            self.model = nn.Sequential(
                nn.Flatten(),
                nn.Linear(28*28, hidden_units),
                nn.ReLU(),
                nn.Linear(hidden_units, 10)
            )
        def forward(self, x):
            return self.model(x)

    model = Net()
    optimizer = optim.Adam(model.parameters(), lr=lr)
    loss_fn = nn.CrossEntropyLoss()

    # Train for a few epochs
    for epoch in range(3):  # Keep it short for comparison
        model.train()
        for images, labels in train_loader:
            optimizer.zero_grad()
            output = model(images)
            loss = loss_fn(output, labels)
            loss.backward()
            optimizer.step()

    # Evaluate
    model.eval()
    correct, total = 0, 0
    with torch.no_grad():
        for images, labels in val_loader:
            outputs = model(images)
            _, predicted = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    accuracy = 100 * correct / total
    results.append((lr, hidden_units, batch_size, accuracy))
    print(f"lr={lr}, hidden={hidden_units}, batch={batch_size} → Val Acc={accuracy:.2f}%")

# Best result
best = max(results, key=lambda x: x[3])
print("\nBest Hyperparameters:", best)
```

```
100%|██████████| 9.91M/9.91M [00:00<00:00, 11.6MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 345kB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 3.19MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 6.47MB/s]
lr=0.001, hidden=64, batch=64 → Val Acc=93.95%
lr=0.001, hidden=64, batch=128 → Val Acc=94.05%
lr=0.001, hidden=128, batch=64 → Val Acc=96.39%
lr=0.001, hidden=128, batch=128 → Val Acc=95.08%
lr=0.01, hidden=64, batch=64 → Val Acc=91.25%
lr=0.01, hidden=64, batch=128 → Val Acc=92.42%
lr=0.01, hidden=128, batch=64 → Val Acc=93.57%
lr=0.01, hidden=128, batch=128 → Val Acc=94.78%

Best Hyperparameters: (0.001, 128, 64, 96.39)
```

```
[ ] #Regression – Synthetic Dataset with MSE
```

```
[ ] from sklearn.datasets import make_regression
    from sklearn.model_selection import train_test_split
    from torch.utils.data import TensorDataset

    # Generate regression data
    X, y = make_regression(n_samples=1000, n_features=10, noise=15)
    X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2)

    # Convert to PyTorch tensors
    X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
    y_train_tensor = torch.tensor(y_train, dtype=torch.float32).view(-1, 1)
    X_val_tensor = torch.tensor(X_val, dtype=torch.float32)
    y_val_tensor = torch.tensor(y_val, dtype=torch.float32).view(-1, 1)

    train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
    val_dataset = TensorDataset(X_val_tensor, y_val_tensor)

    # Hyperparameter grid
    lr_list = [0.01, 0.001]
    hidden_list = [32, 64]

    results_reg = []

    for lr, hidden_units in itertools.product(lr_list, hidden_list):
        train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
        val_loader = DataLoader(val_dataset, batch_size=64)

        class RegNN(nn.Module):
            def __init__(self):
                super().__init__()
                self.net = nn.Sequential(
                    nn.Linear(10, hidden_units),
                    nn.ReLU(),
                    nn.Linear(hidden_units, 1)
                )
            def forward(self, x):
                return self.net(x)
```

```
                nn.ReLU(),
                nn.Linear(hidden_units, 1)
            )
        def forward(self, x):
            return self.net(x)

    model = RegNN()
    optimizer = optim.Adam(model.parameters(), lr=lr)
    loss_fn = nn.MSELoss()

    for epoch in range(3):
        model.train()
        for X_batch, y_batch in train_loader:
            optimizer.zero_grad()
            output = model(X_batch)
            loss = loss_fn(output, y_batch)
            loss.backward()
            optimizer.step()

    # Validation loss
    model.eval()
    val_loss = 0
    with torch.no_grad():
        for X_batch, y_batch in val_loader:
            output = model(X_batch)
            val_loss += loss_fn(output, y_batch).item()

    avg_loss = val_loss / len(val_loader)
    results_reg.append((lr, hidden_units, avg_loss))
    print(f"lr={lr}, hidden={hidden_units} → Val Loss={avg_loss:.4f}")

best_reg = min(results_reg, key=lambda x: x[2])
print("\nBest Regression Hyperparameters:", best_reg)
```

```
lr=0.01, hidden=32 → Val Loss=32989.6680
lr=0.01, hidden=64 → Val Loss=30711.5288
lr=0.001, hidden=32 → Val Loss=36213.8896
lr=0.001, hidden=64 → Val Loss=36123.5786

Best Regression Hyperparameters: (0.01, 64, 30711.52880859375)
```

Conclusion:

Practical 05

Student Name: Poorva Naik
Date of Experiment:
Date of Submission:
PRN No:20220802219

Aim: Develop a Python function (Multivariate Optimization) to compute the Hessian matrix for a given scalar-valued function of multiple variables.

Objectives:

1. Define objective function
2. Define the gradient of the objective function
3. Implement Hessian matrix function using objective and gradient
4. Perform optimization using Newton's method

Software/Tool: Python/Jupyter/Anaconda/Colab

Theory:

In mathematics, the Hessian matrix or Hessian is a square matrix of second-order partial derivatives of a scalar-valued function. It describes the local curvature of a function of many variables. Hessian matrices belong to a class of mathematical structures that involve second order derivatives. They are often used in machine learning and data science algorithms for optimizing a function of interest. The Hessian matrix is a mathematical tool used to calculate the curvature of a function at a certain point in space.

The formula for Hessian Matrix is given as

$$H(f) = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{pmatrix} \qquad \mathbf{H}f = \begin{bmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} & \frac{\partial^2 f}{\partial x \partial z} & \cdots \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y^2} & \frac{\partial^2 f}{\partial y \partial z} & \cdots \\ \frac{\partial^2 f}{\partial z \partial x} & \frac{\partial^2 f}{\partial z \partial y} & \frac{\partial^2 f}{\partial z^2} & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix}$$

Algorithm

1. **Start**
2. **Import Required Libraries**
   - Use numpy, pandas, matplotlib, seaborn, and autograd for numerical operations and visualization.
3. **Load the Dataset**
   - Read the dataset (e.g., creditcard.csv) into a DataFrame using pandas.
4. **Preprocess the Data**
   - Select only numeric columns using select_dtypes().
   - Drop any rows with missing values.
   - Normalize the data (standardization) for numerical stability using Z-score normalization:
   - $X\_scaled = (X - \mu) / \sigma$
5. **Select Features**
   - Choose three features from the dataset (e.g., V9, V10, V11) for further analysis.
6. **Define the Function to Analyze**
   - Create a mathematical function involving the selected features:

$$f(a, b) = e^a + \log(b^2 + 1) + 2ab$$

7. **Compute the Hessian Matrix**
   - Use autograd.hessian() to compute the second-order partial derivatives of the function with respect to selected features.
   - Evaluate the Hessian at the mean point of the chosen features.
8. **Visualize the Hessian Matrix**
   - Use seaborn.heatmap() to display the Hessian matrix.
   - Annotate the heatmap with values for easy interpretation.
9. **Create a Contour Plot**
   - Plot a 2D contour using two of the selected features.
   - Use the same function definition for surface computation.
   - Highlight the mean data point on the plot.

Explanation of the Algorithm for Hessian Matrix Analysis

This algorithm demonstrates how to analyze the **curvature** of a multivariable function using the **Hessian matrix**, which helps in optimization, gradient descent tuning, and understanding feature interactions.

1. **Why Normalize?**
   Normalization brings all values to a similar scale, which avoids numeric instability while computing derivatives.
2. **What is the Hessian Matrix?**
   The Hessian is a square matrix of second-order partial derivatives. It tells us how each input variable interacts with others in terms of curvature of a function.

3. **Function Interpretation**
   The chosen function includes exponential, logarithmic, and product terms—mimicking real-world nonlinear behavior in data.

4. **Heatmap and Contour Plot**
   - The heatmap visually shows where the function curves most strongly.
   - The contour plot gives a surface view of how function values change with input variables.

Program code, Results and output

```python
import sympy as sp

def compute_hessian(f, variables):

    # Compute gradient (first-order partial derivatives)
    gradient = [sp.diff(f, var) for var in variables]

    # Compute Hessian matrix (second-order partial derivatives)
    hessian = sp.Matrix([[sp.diff(gradient[i], var) for var in variables]
                         for i in range(len(variables))])

    return hessian

if __name__ == "__main__":
    # Define symbolic variables
    x, y, z = sp.symbols('x y z')

    # Define the scalar-valued multivariate function
    f = 2*x**2 + 3*x*y + 10*y**2 + 8*y*z + 2*z**3

    # List of variables
    variables = [x, y, z]

    # Compute the Hessian matrix
    hessian = compute_hessian(f, variables)

    # Display result
    print("Hessian matrix:")
    sp.pprint(hessian)
```

```
Hessian matrix:
⎡4  3    0  ⎤
⎢           ⎥
⎢3  20   8  ⎥
⎢           ⎥
⎣0  8   12⋅z⎦
```

```python
import numpy as np
from scipy.linalg import eigh  # For eigenvalues

# 1. Define the objective function
def objective_function(x):
    # Example: f(x, y, z) = 2x^2 + 3xy + 10y^2 + 8yz + 2z^3
    return 2*x[0]**2 + 3*x[0]*x[1] + 10*x[1]**2 + 8*x[1]*x[2] + 2*x[2]**3

# 2 & 3. Compute Hessian matrix using finite differences
def compute_hessian_fd(f, x, h=1e-5):
    n = len(x)
    hessian = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            x1 = x.copy(); x1[i] += h; x1[j] += h
            x2 = x.copy(); x2[i] += h; x2[j] -= h
            x3 = x.copy(); x3[i] -= h; x3[j] += h
            x4 = x.copy(); x4[i] -= h; x4[j] -= h
            hessian[i, j] = (f(x1) - f(x2) - f(x3) + f(x4)) / (4*h**2)
    return hessian

# 4, 5, 6. Analyze Hessian
def analyze_hessian(hessian):
    rounded_hessian = np.round(hessian, 3)
    print("\nHessian Matrix:")
    print(rounded_hessian)

    # Symmetry check
    is_symmetric = np.allclose(hessian, hessian.T)
    print("Is Symmetric:", is_symmetric)

    # Eigenvalues and definiteness
    eigenvalues = eigh(hessian, eigvals_only=True)
    print("Eigenvalues:", np.round(eigenvalues, 5))

    is_positive_definite = np.all(eigenvalues > 0)
    print("Is Positive Definite:", is_positive_definite)

    # ✅ Determinant
    determinant = np.linalg.det(hessian)
    print("Determinant of Hessian:", round(determinant, 5))
```


```python
    print("Is Positive Definite:", is_positive_definite)

    # ✅ Determinant
    determinant = np.linalg.det(hessian)
    print("Determinant of Hessian:", round(determinant, 5))
```


```python
    # Interpretation
    if is_positive_definite:
        print("→ The function has a local minimum at the point.")
    elif np.all(eigenvalues < 0):
        print("→ The function has a local maximum at the point.")
    else:
        print("→ The function has a saddle point at the point.")

# Main function
if __name__ == "__main__":
    point = np.array([1.0, 1.0, 1.0])  # Evaluation point
    hessian = compute_hessian_fd(objective_function, point)
    analyze_hessian(hessian)
```

```
Hessian Matrix:
[[ 4.  3.  0.]
 [ 3. 20.  8.]
 [ 0.  8. 12.]]
Is Symmetric: True
Eigenvalues: [ 3.07699  7.67034 25.2527 ]
Is Positive Definite: True
Determinant of Hessian: 596.00299
→ The function has a local minimum at the point.
```

Conclusion:

Practical 06

Student Name: Poorva Naik
Date of Experiment:
Date of Submission:
PRN No:20220802219

Aim: Implement Bayesian Neural Network

Objectives:

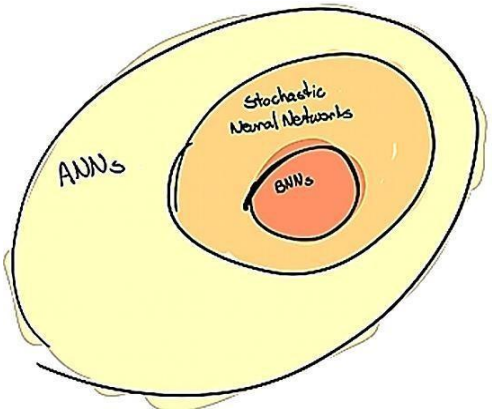
1. Designing Bayesian Neural Network

Software/Tool: Python/Jupyter/Anaconda/Colab

Dataset link : link

Theory:

Deep learning models tend to overfitting and have several problems in establishing the uncertainties of their predictions. Bayesian Neural Networks are a specific type of neural networks trained in the light of the Bayesian paradigm, being capable to quantify uncertainty associated with the underlying processes. This approach can be helpful in applications where model failures are especially costly, e.g., autonomous driving, medical diagnosis or finance.

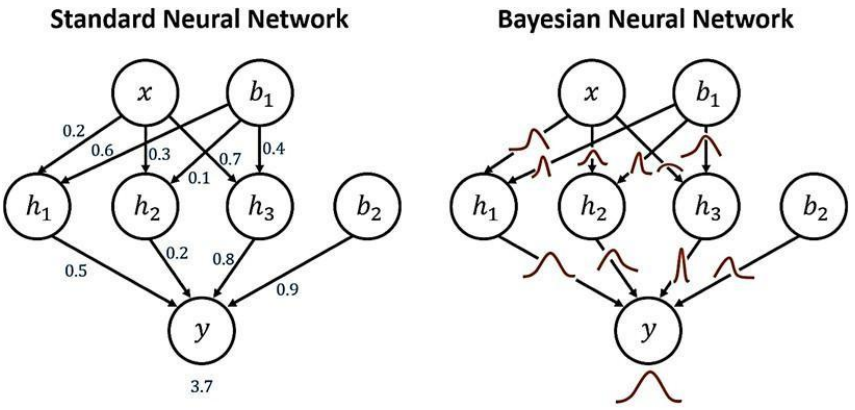


1. SNN focuses on optimization while BNN focuses on marginalization. Optimization would find one optimal point to represent a weight while marginalization would treat each weight as a variable and find its distribution.
2. **Estimate —** The estimate of the parameters for SNN would be maximum likelihood estimators (MLE) while for BNN; the estimate would be rather maximum a posteriori (MAP) or predictive distribution.
3. **Method —** Basically, SNN would use differentiation to find the optimal value such as gradient descent. In BNN, since the sophisticated integrals are hard to determine, scientists or researchers would always rely on Markov Chain Monte Carlos (MCMC), Variational…

Design of BNN

The procedure to design a BNN can be divided into the steps:

- Choice of a **functional model: *y = Φ(x)*** (architecture).
- Choice of a **stochastic model: *p(θ)*** for model parametrization and ***p(y|x, θ)*** for confidence of the model.



Functional model

- To obtain the posterior distribution for parameters, given our data *D = {Dx, Dy}* with training inputs and labels, respectively.
- Due to the complexity of the posterior — especially because to the evidence integral term —, computing this in a standard way is in general intractable.

In short, to obtain estimates in a model through BNNs, we need **a stochastic model containing our priors** (and for the variational case, a variational posterior), **a functional model for the network** and the **training data**.

- From there, **training is done using one of the available sampling methods, so that we can infer the posterior for the parameters.**
- From the posterior, **we can evaluate the reliability of our model by obtaining its marginal**, encoded from an estimator given by the mean of the models and uncertainty given by their respective covariance matrix.
- BNNs have been used in many fields to quantify uncertainty, *e.g.*, in computer vision, network traffic monitoring, aviation, civil engineering, hydrology, astronomy, electronics, and medicine

Algorithm

Start

1. Load the breast cancer dataset (e.g., using load_breast_cancer() from sklearn).
2. Extract relevant features:
    - Select the data matrix as input features **X**.
    - Select the diagnosis labels as the target variable **y**.
3. Split the dataset into training and testing sets using train_test_split().
4. Define a NaiveBayesClassifier class with:
    - fit() method to calculate:
        - Prior probabilities for each class.
        - Mean and standard deviation for each feature per class.
    - predict() method to:
        - Calculate the likelihood using the normal distribution PDF.
        - Compute posterior probability and select the class with the highest value.

5. Train the classifier on the training data using the fit() method.
6. Predict the labels for the test data using the predict() method.
7. Calculate and display the accuracy of predictions.
8. Create a function predict_cancer() to classify a new sample using the trained model.
9. Predict and display if a new sample is “Cancer” or “No Cancer”.

Explanation of the Algorithm for Naive Bayes Classifier on Breast Cancer Dataset

This algorithm uses probabilistic principles to classify whether a tumor is benign or malignant based on various physical measurements of cell nuclei.

1. **Input**
    - The dataset includes measurements like radius, perimeter, smoothness, etc.
    - Each data point is labeled as either 0 (Malignant) or 1 (Benign).
2. **Start**
    - We begin the process of building and evaluating a Naive Bayes classifier.
3. **Load the Dataset**
    - The breast cancer dataset is loaded directly from sklearn.datasets.
    - It includes numerical features and binary labels.
4. **Feature Extraction**
    - X: All the numeric features of each tumor.
    - y: The corresponding diagnosis (0 or 1).
5. **Train-Test Split**
    - The data is split into training and testing sets for model evaluation.
6. **Model Definition**
    - The NaiveBayesClassifier uses Gaussian Naive Bayes logic.
    - It calculates the probability of each class given a set of input features using Bayes’ theorem.
7. **Training Phase**
    - For each class, calculate the mean and standard deviation for each feature.
    - Also calculate the prior probability of each class (based on frequency).
8. **Prediction Phase**
    - For each test sample, calculate the probability it belongs to each class.
    - The class with the highest probability is selected as the prediction.
9. **Accuracy Calculation**
    - The predictions are compared with actual test labels to compute accuracy.
10. **Cancer Prediction Function**

- A separate function takes a new sample and predicts its label as “Cancer” or “No Cancer”.

11. **End**

- The model is ready for use in prediction and analysis tasks.

Program code, Results and output

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical

# Load MNIST data
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train = x_train.reshape(-1, 28*28) / 255.0
x_test = x_test.reshape(-1, 28*28) / 255.0
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# BNN parameters
input_size = 784
hidden_size = 128
output_size = 10
learning_rate = 0.001
epochs = 50
batch_size = 128

# Weight and bias initializations
np.random.seed(42)
W1_mu = np.random.randn(input_size, hidden_size) * 0.1
W1_sigma = np.ones((input_size, hidden_size)) * 0.1
b1_mu = np.zeros(hidden_size)
b1_sigma = np.ones(hidden_size) * 0.1

W2_mu = np.random.randn(hidden_size, output_size) * 0.1
W2_sigma = np.ones((hidden_size, output_size)) * 0.1
b2_mu = np.zeros(output_size)
b2_sigma = np.ones(output_size) * 0.1

def relu(x):
    return np.maximum(0, x)

def softmax(x):
    e_x = np.exp(x - np.max(x, axis=1, keepdims=True))
    return e_x / np.sum(e_x, axis=1, keepdims=True)

def cross_entropy(predictions, targets):
```

```
def cross_entropy(predictions, targets):
    return -np.mean(np.sum(targets * np.log(predictions + 1e-9), axis=1))

def accuracy(predictions, targets):
    return np.mean(np.argmax(predictions, axis=1) == np.argmax(targets, axis=1))

def kl_divergence(mu, sigma, prior_std=0.1):
    return 0.5 * np.sum(np.log(sigma ** 2 + 1e-8) - np.log(prior_std ** 2) + (prior_std ** 2 + mu ** 2) / (sigma ** 2 + 1e-8) - 1)

loss_curve = []
accuracy_curve = []

for epoch in range(epochs):
    # Save parameters before training step
    W1_mu_before = W1_mu.copy()
    W2_mu_before = W2_mu.copy()
    b1_mu_before = b1_mu.copy()
    b2_mu_before = b2_mu.copy()

    indices = np.arange(x_train.shape[0])
    np.random.shuffle(indices)
    x_train = x_train[indices]
    y_train = y_train[indices]

    epoch_loss = 0
    epoch_acc = 0
    num_batches = x_train.shape[0] // batch_size

    for i in range(num_batches):
        start, end = i * batch_size, (i + 1) * batch_size
        x_batch = x_train[start:end]
        y_batch = y_train[start:end]

        W1 = W1_mu + W1_sigma * np.random.randn(*W1_mu.shape)
        b1 = b1_mu + b1_sigma * np.random.randn(*b1_mu.shape)
        W2 = W2_mu + W2_sigma * np.random.randn(*W2_mu.shape)
        b2 = b2_mu + b2_sigma * np.random.randn(*b2_mu.shape)

        z1 = x_batch.dot(W1) + b1
        a1 = relu(z1)
        z2 = a1.dot(W2) + b2
```

```
        z1 = x_batch.dot(W1) + b1
        a1 = relu(z1)
        z2 = a1.dot(W2) + b2
        y_pred = softmax(z2)

        loss = cross_entropy(y_pred, y_batch)
        kl = kl_divergence(W1_mu, W1_sigma) + kl_divergence(W2_mu, W2_sigma)
        kl += kl_divergence(b1_mu, b1_sigma) + kl_divergence(b2_mu, b2_sigma)
        total_loss = loss + kl / x_train.shape[0]

        acc = accuracy(y_pred, y_batch)
        epoch_loss += total_loss
        epoch_acc += acc

        dL_dz2 = y_pred - y_batch
        dL_dW2 = a1.T.dot(dL_dz2) / batch_size
        dL_db2 = np.mean(dL_dz2, axis=0)

        dL_da1 = dL_dz2.dot(W2.T)
        dL_dz1 = dL_da1 * (z1 > 0)
        dL_dW1 = x_batch.T.dot(dL_dz1) / batch_size
        dL_db1 = np.mean(dL_dz1, axis=0)

        W1_mu -= learning_rate * dL_dW1
        b1_mu -= learning_rate * dL_db1
        W2_mu -= learning_rate * dL_dW2
        b2_mu -= learning_rate * dL_db2

    epoch_loss /= num_batches
    epoch_acc /= num_batches
    loss_curve.append(epoch_loss)
    accuracy_curve.append(epoch_acc)

    # Test Accuracy
    z1_test = x_test.dot(W1) + b1
    a1_test = relu(z1_test)
    z2_test = a1_test.dot(W2) + b2
    y_pred_test = softmax(z2_test)
    test_acc = accuracy(y_pred_test, y_test)
```

```
    y_pred_test = softmax(z2_test)
    test_acc = accuracy(y_pred_test, y_test)

    # === Print epoch stats ===
    print(f"Epoch {epoch+1}")
    print("Weights and Biases before training step:")
    print(f"W1 mean: {W1_mu_before.mean()} W2 mean: {W2_mu_before.mean()}")
    print(f"b1 mean: {b1_mu_before.mean()} b2 mean: {b2_mu_before.mean()}")
    print(f"Training Accuracy: {epoch_acc:.4f}, Loss: {epoch_loss:.4f}")
    print("Parameter changes (mean deltas):")
    print(f"W1_mu change: {(W1_mu - W1_mu_before).mean()}")
    print(f"W2_mu change: {(W2_mu - W2_mu_before).mean()}")
    print(f"b1_mu change: {(b1_mu - b1_mu_before).mean()}")
    print(f"b2_mu change: {(b2_mu - b2_mu_before).mean()}")
    print(f"Test Accuracy: {test_acc:.4f}")
    print()


# Plotting
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(range(1, epochs + 1), loss_curve, marker='o', label='Training Loss')
plt.title("Loss Curve")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(range(1, epochs + 1), accuracy_curve, marker='o', label='Training Accuracy')
plt.title("Accuracy Curve")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()

plt.tight_layout()
plt.show()
```

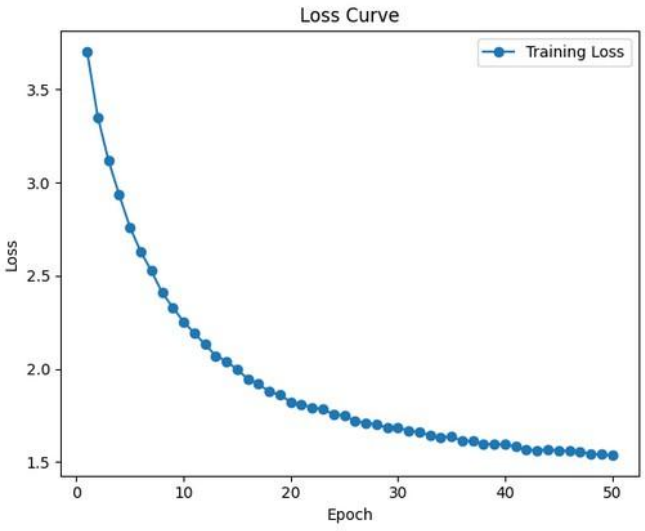
```
Epoch 1
Weights and Biases before training step:
W1 mean: 8.564344779611287e-05 W2 mean: 0.0006640836133769576
b1 mean: 0.0 b2 mean: 0.0
Training Accuracy: 0.1256, Loss: 3.7042
Parameter changes (mean deltas):
W1_mu change: -0.00032299546386993646
W2_mu change: 2.612927235690066e-18
b1_mu change: -0.002359942037773572
b2_mu change: 2.4286128663675298e-18
Test Accuracy: 0.1201

Epoch 2
Weights and Biases before training step:
W1 mean: -0.00023735201607382372 W2 mean: 0.0006640836133769598
b1 mean: -0.002359942037773572 b2 mean: 2.4286128663675298e-18
Training Accuracy: 0.1743, Loss: 3.3513
Parameter changes (mean deltas):
W1_mu change: -0.00021186605435993876
W2_mu change: 3.4748679628160414e-18
b1_mu change: -0.0015580715623866733
b2_mu change: 2.5153490401663702e-18
Test Accuracy: 0.2810

Epoch 3
Weights and Biases before training step:
W1 mean: -0.0004492180704337624 W2 mean: 0.0006640836133769647
b1 mean: -0.003918013600160245 b2 mean: 5.204170427930421e-18
Training Accuracy: 0.2287, Loss: 3.1183
Parameter changes (mean deltas):
W1_mu change: -0.00015151592096488103
W2_mu change: 4.771844811651827e-18
b1_mu change: -0.0010897725837922467
b2_mu change: 1.5612511283791264e-18
Test Accuracy: 0.2436

Epoch 4
Weights and Biases before training step:
W1 mean: -0.0006007339913986436 W2 mean: 0.0006640836133769689
b1 mean: -0.0050077861839524914 b2 mean: 6.7654215563095475e-18
Training Accuracy: 0.2890, Loss: 2.9332
Parameter changes (mean deltas):
W1_mu change: -0.0001130304970542763
```

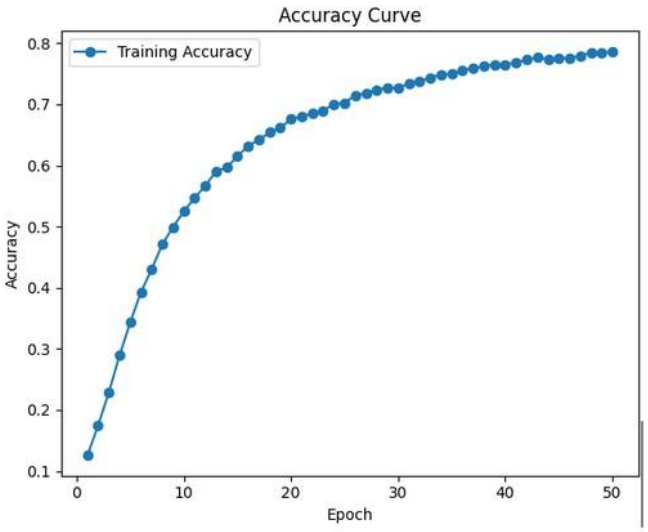

```
Epoch 4
Weights and Biases before training step:
W1 mean: -0.0006007339913986436 W2 mean: 0.0006640836133769689
b1 mean: -0.0050077861839524914 b2 mean: 6.7654215563095475e-18
Training Accuracy: 0.2890, Loss: 2.9332
Parameter changes (mean deltas):
W1_mu change: -0.0001130304970542763
```


```
W2_mu change: -2.392021043046144e-19
b1_mu change: -0.000757041408042481
b2_mu change: -3.816391647148976e-18
Test Accuracy: 0.3385

Epoch 5
Weights and Biases before training step:
W1 mean: -0.0007137644884529198 W2 mean: 0.0006640836133769686
b1 mean: -0.005764827591994972 b2 mean: 3.0357660829594124e-18
Training Accuracy: 0.3434, Loss: 2.7602
Parameter changes (mean deltas):
W1_mu change: -6.846304451609129e-05
W2_mu change: 2.087089182034596e-18
b1_mu change: -0.0004328310238358064
b2_mu change: 5.204170427930421e-18
Test Accuracy: 0.3977

Epoch 6
Weights and Biases before training step:
W1 mean: -0.0007822275329690111 W2 mean: 0.0006640836133769704
b1 mean: -0.006197658615830779 b2 mean: 7.979727989493312e-18
Training Accuracy: 0.3930, Loss: 2.6290
Parameter changes (mean deltas):
W1_mu change: -4.7082847156363276e-05
W2_mu change: -4.052205619664573e-18
b1_mu change: -0.0002510986719287154
b2_mu change: 5.204170427930421e-18
Test Accuracy: 0.4565

Epoch 7
Weights and Biases before training step:
W1 mean: -0.0008293103801253745 W2 mean: 0.0006640836133769668
b1 mean: -0.006448757287759494 b2 mean: 1.3444106938820255e-17
Training Accuracy: 0.4302, Loss: 2.5255
Parameter changes (mean deltas):
W1_mu change: -3.381723068636006e-05
```

```
Epoch 7
Weights and Biases before training step:
W1 mean: -0.0008293103801253745 W2 mean: 0.0006640836133769668
b1 mean: -0.0064487572877594945 b2 mean: 1.3444106938820255e-17
Training Accuracy: 0.4302, Loss: 2.5255
Parameter changes (mean deltas):
W1_mu change: -3.381723068636006e-05
W2_mu change: 1.7984203536103303e-18
b1_mu change: -0.0001343664117203706
b2_mu change: 1.3921155894713878e-17
Test Accuracy: 0.4873

Epoch 8
Weights and Biases before training step:
W1 mean: -0.0008631276108117343 W2 mean: 0.0006640836133769683
b1 mean: -0.006583123699479865 b2 mean: 2.654126918244515e-17
Training Accuracy: 0.4712, Loss: 2.4098
Parameter changes (mean deltas):
W1_mu change: -1.66240498984315e-05
W2_mu change: -6.857578740970816e-19
b1_mu change: -9.04821146789796e-06
b2_mu change: 1.3010426069826053e-17
Test Accuracy: 0.4033

Epoch 9
Weights and Biases before training step:
W1 mean: -0.0008797516607101661 W2 mean: 0.0006640836133769672
b1 mean: -0.0065921719109477625 b2 mean: 3.9638431426070045e-17
Training Accuracy: 0.4992, Loss: 2.3288
Parameter changes (mean deltas):
W1_mu change: -1.1768308406882931e-05
W2_mu change: -4.527899322842588e-18
b1_mu change: 2.7253402517063427e-05
b2_mu change: -2.5587171270657906e-18
Test Accuracy: 0.5888

Epoch 10
Weights and Biases before training step:
W1 mean: -0.000891519969117049 W2 mean: 0.0006640836133769634
b1 mean: -0.0065649185084307 b2 mean: 3.699297812520541e-17
Training Accuracy: 0.5249, Loss: 2.2533
Parameter changes (mean deltas):
W1_mu change: -5.459834223478475e-06
```

```
Epoch 48
Weights and Biases before training step:
W1 mean: -0.0012709666954707013 W2 mean: 0.0006640836133770104
b1 mean: -0.00669232592273342 b2 mean: 5.204170427930421e-19
Training Accuracy: 0.7845, Loss: 1.5431
Parameter changes (mean deltas):
W1_mu change: -1.4454798902260694e-05
W2_mu change: 1.0369377340287145e-18
b1_mu change: -5.350650873774562e-05
b2_mu change: -2.656295322589486e-17
Test Accuracy: 0.7882

Epoch 49
Weights and Biases before training step:
W1 mean: -0.0012854214943729622 W2 mean: 0.0006640836133770122
b1 mean: -0.006745832431471165 b2 mean: -2.673642557349254e-17
Training Accuracy: 0.7840, Loss: 1.5401
Parameter changes (mean deltas):
W1_mu change: -1.4186986645515606e-05
W2_mu change: 4.189255550530319e-18
b1_mu change: -5.6452899608018966e-05
b2_mu change: -5.37330596683816e-17
Test Accuracy: 0.7875

Epoch 50
Weights and Biases before training step:
W1 mean: -0.001299608481018478 W2 mean: 0.0006640836133770154
b1 mean: -0.006802285331079184 b2 mean: -8.133684697986255e-17
Training Accuracy: 0.7867, Loss: 1.5351
Parameter changes (mean deltas):
W1_mu change: -1.5286378185798184e-05
W2_mu change: 6.753732501637438e-18
b1_mu change: -5.8357717986205616e-05
b2_mu change: -2.0664893407573714e-17
Test Accuracy: 0.8151
```


Loss Curve
Training Loss
Loss
Epoch
3.5
3.0
2.5
2.0
1.5
0
10
20
30
40
50


Accuracy Curve
Training Accuracy
Accuracy
Epoch
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
10
20
30
40
50

Conclusion:

Practical 07

Student Name: Poorva Naik
Date of Experiment:
Date of Submission:
PRN No: 20220802219

Aim: Implement Regularization in neural Network

Objectives:

1. Early stopping
2. L1 and L2 regularization
3. Data augmentation
4. Addition of noise
5. Dropout

Software/Tool: Python/Jupyter/Anaconda/Colab

Dataset link : link

Theory:

1) Early stopping

- Early stopping is one of the simplest and most intuitive regularization techniques.
- It involves stopping the training of the neural network at an earlier epoch;
- As you train the neural network over many epochs, the training error decreases.
- If the training error becomes too low and reaches arbitrarily close to zero, then the network is sure to overfit on the training dataset.
- Such a neural network is a high variance model that performs badly on test data that it has never seen before despite its near-perfect performance on the training samples.
- Therefore, heuristically, if we can prevent the training loss from becoming arbitrarily low, the model is less likely to overfit on the training dataset, and will generalize better.

2) Data Augmentations

- Data augmentation is a regularization technique that helps a neural network generalize better by exposing it to a more diverse set of training examples.
- As deep neural networks require a large training dataset, data augmentation is also helpful when we have insufficient data to train a neural network.
- Let's take the example of image data augmentation. Suppose we have a dataset with N training examples across C classes.
- We can apply certain transformations to these N images to construct a larger dataset.

2) L1 and L2 Regularization

a) Lp

In general, Lp norms (for p>=1) penalize larger weights. They force the norm of the weight vector to stay sufficiently small.

b) L1 norm

When p=1, we get L1 norm, the sum of the absolute values of the components in the vector

c) L2 norm

When p=2, we get L2 norm, the Euclidean distance of the point from the origin in n-dimensional vector space

4) Addition of Noise

- Another regularization approach is the addition of noise. You can add noise to the input, the output labels, or the gradients of the neural network.

5) Dropout

- Dropout involves dropping neurons in the hidden layers and (optionally) the input layer.
- During training, each neuron is assigned a "dropout" probability, like 0.5.
- With a dropout of 0.5, there's a 50% chance of each neuron participating in training within each training batch.
- This results in slightly different network architecture for each batch.
- It is equivalent to training different neural networks on different subsets of the training data.

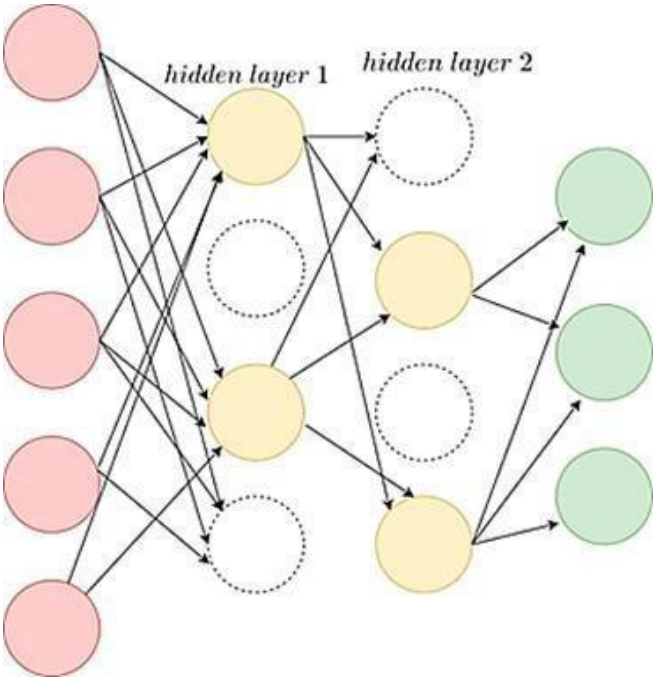


Figure: A Neural Network with dropout

Algorithm

Start

1. Import necessary libraries (torch, torchvision, etc.).
2. Apply data augmentation and normalization using transforms:
    - Random rotation
    - Horizontal flipping
    - Conversion to tensor and normalization
3. Load the MNIST dataset with defined transformations.
4. Split the training data into training and validation sets (80-20).
5. Create data loaders for training, validation, and test sets.
6. Define the neural network architecture with:
    - Two hidden layers with ReLU activation
    - Dropout applied after the first hidden layer
    - Final output layer with 10 neurons for classification
7. Initialize the model and optimizer (Adam) with **L2 regularization** using weight_decay.
8. Define the loss function as CrossEntropyLoss.
9. Set up **early stopping** mechanism using validation loss and a defined patience.
10. Start the training loop for a defined number of epochs:

- For each batch:
  - Forward pass to compute predictions
  - Compute loss (add L1 regularization manually)
  - Backward pass and optimizer step
- Validate the model on the validation set
- Apply early stopping if validation loss doesn't improve

11. After training, evaluate the model on the test set.
12. Compute and display the test accuracy.

Explanation of the Algorithm for Regularized Neural Network on MNIST Dataset

This algorithm implements a neural network to classify handwritten digits using the MNIST dataset. It incorporates regularization and early stopping to prevent overfitting and improve model performance.

1. **Input**
   - The MNIST dataset contains 28×28 grayscale images of digits from 0 to 9, with labels indicating the actual digit.
2. **Start**
   - Initialize the process of training a neural network with regularization techniques.
3. **Data Augmentation and Normalization**
   - Transform the images by rotating and flipping to increase variation.
   - Normalize images to have zero mean and unit variance.
4. **Loading and Splitting Dataset**
   - Load the MNIST dataset and divide it into training, validation, and testing subsets.
5. **Neural Network Design**
   - Build a fully connected feedforward neural network:
     - Input: 784 neurons (28×28)
     - Hidden layers: 512 and 256 neurons with ReLU
     - Output: 10 neurons (for digit classes)
   - Use **Dropout** after the first hidden layer to randomly disable 50% of the neurons during training, reducing overfitting.
6. **Model Training with Regularization**
   - Use **L2 regularization** via weight_decay in the optimizer to penalize large weights.
   - Apply **L1 regularization** by adding the sum of absolute weights to the loss function.
   - These techniques force the model to prefer smaller, more generalizable weights.
7. **Early Stopping**
   - Monitor validation loss after each epoch.
   - Stop training if validation loss doesn't improve for 5 consecutive epochs.
8. **Model Evaluation**

- After training, run inference on the test dataset.
- Compute overall accuracy by comparing predicted labels with actual labels

Dataset: MNIST Dataset

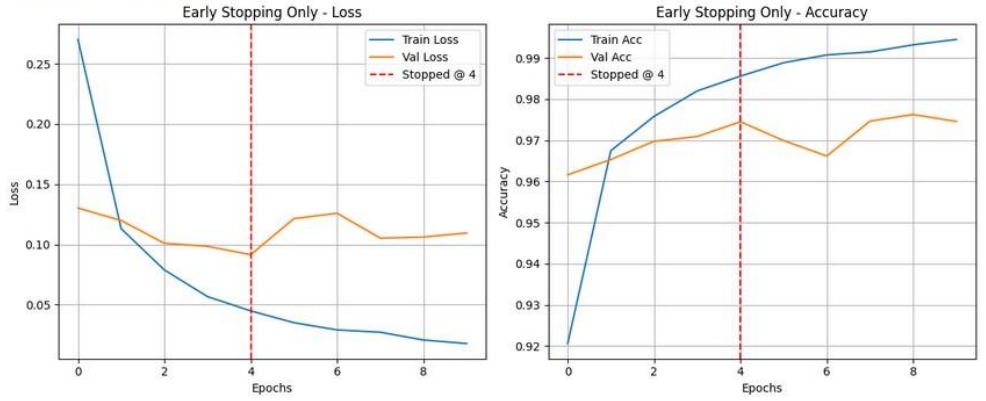
- Type: Image classification (handwritten digits)
- Number of Classes: 10 (digits 0 through 9)
- Image Size: 28 × 28 pixels (grayscale images)
- Channels: 1 (since it's grayscale)
- Total Samples:
- Training Set: 60,000 images
- Test Set: 10,000 images

Program code, Results and output

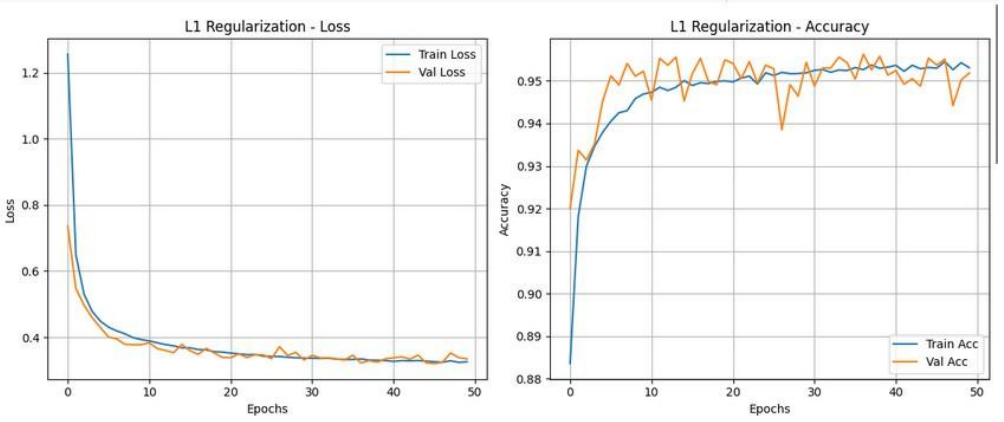
```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Dropout
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.regularizers import l1, l2
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical

# Load MNIST
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Normalize images
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# One-hot encode labels
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Helper function to plot training history
def plot_history(history, title, stop_epoch=None):
    plt.figure(figsize=(12, 5))

    # Loss Plot
    plt.subplot(1, 2, 1)
    plt.plot(history.history['loss'], label='Train Loss')
    plt.plot(history.history['val_loss'], label='Val Loss')
    if stop_epoch is not None:
        plt.axvline(stop_epoch, color='r', linestyle='--', label=f'Stopped @ {stop_epoch}')
    plt.title(f"{title} - Loss")
    plt.xlabel("Epochs")
    plt.ylabel("Loss")
    plt.legend()
    plt.grid(True)
```

```
    # Accuracy Plot
    plt.subplot(1, 2, 2)
    plt.plot(history.history['accuracy'], label='Train Acc')
    plt.plot(history.history['val_accuracy'], label='Val Acc')
    if stop_epoch is not None:
        plt.axvline(stop_epoch, color='r', linestyle='--', label=f'Stopped @ {stop_epoch}')
    plt.title(f"{title} - Accuracy")
    plt.xlabel("Epochs")
    plt.ylabel("Accuracy")
    plt.legend()
    plt.grid(True)

    plt.tight_layout()
    plt.show()

# ---------- 1. Early Stopping Only -------------
model_es = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(128, activation='relu'),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])
model_es.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

es_callback = EarlyStopping(patience=5, restore_best_weights=True)
history_es = model_es.fit(x_train, y_train, validation_split=0.2, epochs=50, callbacks=[es_callback], verbose

best_epoch = np.argmin(history_es.history['val_loss'])
plot_history(history_es, "Early Stopping Only", stop_epoch=best_epoch)
```

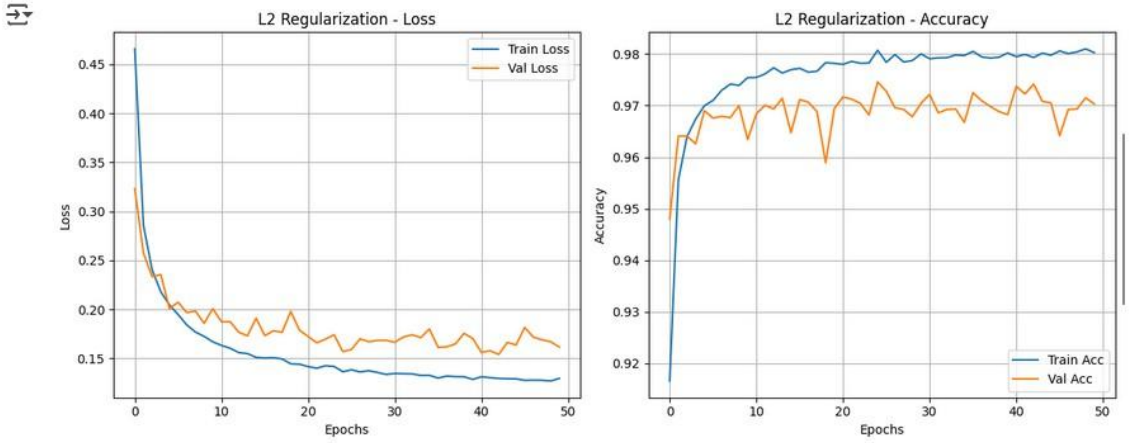
```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
/usr/local/lib/python3.11/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an
  super().__init__(**kwargs)
```



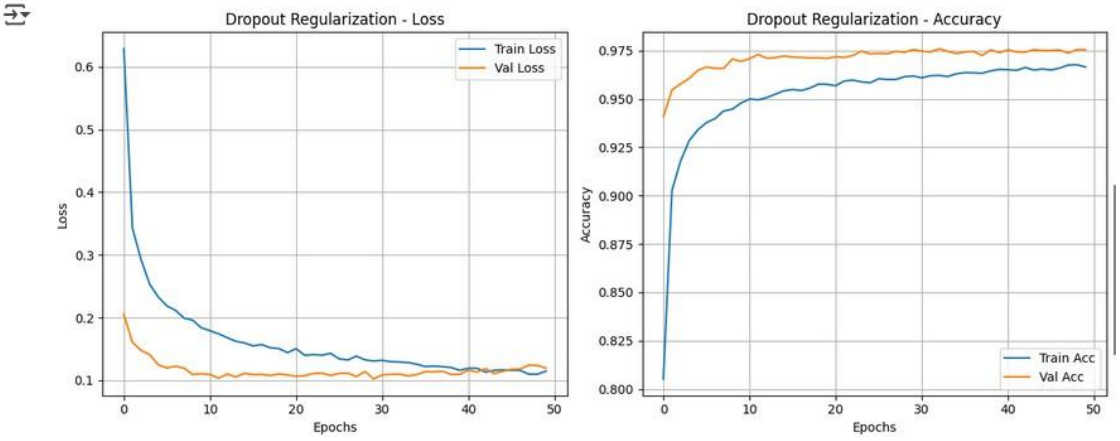

```
# ---------- 2. L1 Regularization -------------
model_l1 = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(128, activation='relu', kernel_regularizer=l1(0.001)),
    Dense(64, activation='relu', kernel_regularizer=l1(0.001)),
    Dense(10, activation='softmax')
])
model_l1.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

history_l1 = model_l1.fit(x_train, y_train, validation_split=0.2, epochs=50, verbose=0)
plot_history(history_l1, "L1 Regularization")
```



```
# ----------- 3. L2 Regularization -------------
model_l2 = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(128, activation='relu', kernel_regularizer=l2(0.001)),
    Dense(64, activation='relu', kernel_regularizer=l2(0.001)),
    Dense(10, activation='softmax')
])
model_l2.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

history_l2 = model_l2.fit(x_train, y_train, validation_split=0.2, epochs=50, verbose=0)
plot_history(history_l2, "L2 Regularization")
```




```
# ----------- 4. Dropout -------------
model_dropout = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(64, activation='relu'),
    Dropout(0.5),
    Dense(10, activation='softmax')
])
model_dropout.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

history_dropout = model_dropout.fit(x_train, y_train, validation_split=0.2, epochs=50, verbose=0)
plot_history(history_dropout, "Dropout Regularization")
```



Conclusion:

Practical 08

Student Name: Poorva Naik
Date of Experiment:
Date of Submission:
PRN No:20220802219

Aim: Implement EM (Expectation Maximization) Algorithm

Objectives:

1. EM (Expectation Maximization) Algorithm

Software/Tool: Python/Jupyter/Anaconda/Colab

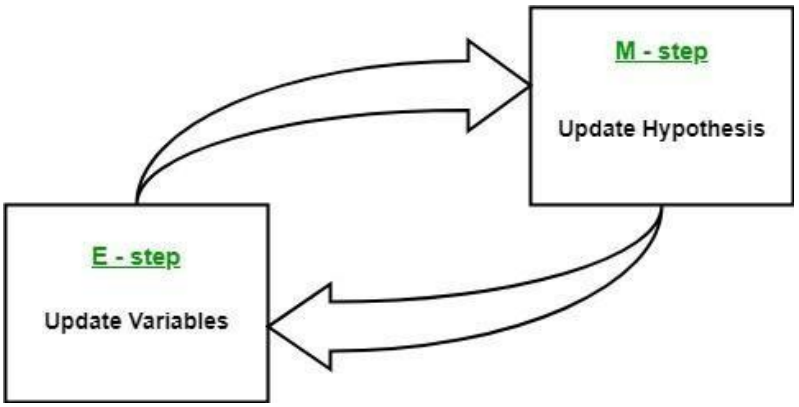
Dataset link :link

Theory:

- The Expectation-Maximization algorithm can be used to handle situations where variables are partially observable.
- When certain variables are observable, we can use those instances to learn and estimate their values. Then, we can predict the values of these variables in instances when it is not observable.
- EM algorithm is applicable to latent variables, which are variables that are not directly observable but are inferred from the values of other observed variables.
- By leveraging the known general form of the probability distribution governing these latent variables, the EM algorithm can predict their values.
- It provides a framework to find the local maximum likelihood parameters of a statistical model and infer latent variables in cases where data is missing or incomplete.

The Expectation-Maximization (EM) algorithm is an iterative optimization method that combines different unsupervised machine learning algorithms to find maximum likelihood or maximum posterior estimates of parameters in statistical models that involve unobserved latent variables.

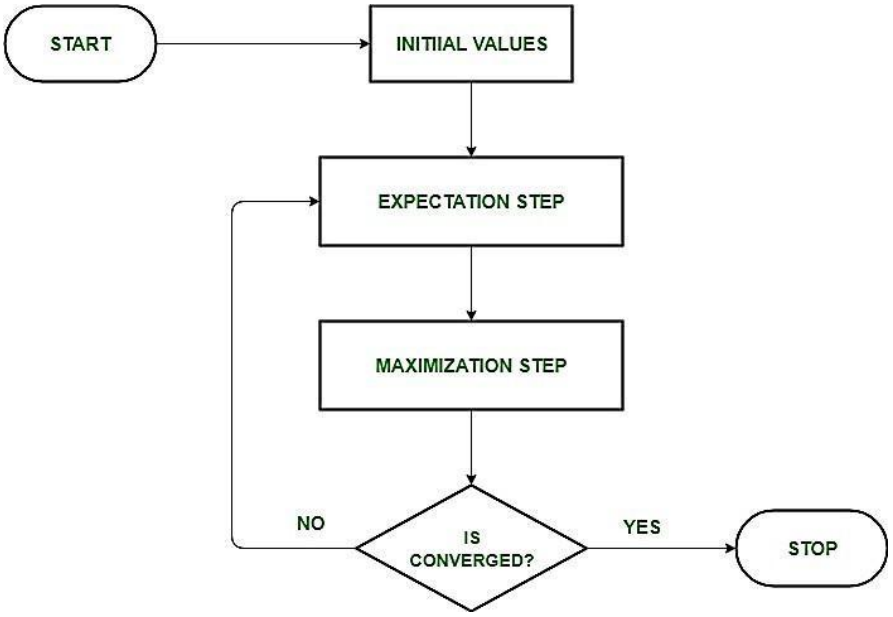
The EM algorithm is commonly used for latent variable models and can handle missing data. It consists of an estimation step (E-step) and a maximization step (M-step), forming an iterative process to improve model fit.



- Posterior Probability: In the context of Bayesian inference, the EM algorithm can be extended to estimate the maximum a posteriori (MAP) estimates, where the posterior probability of the parameters is calculated based on the prior distribution and the likelihood function.
- Expectation (E) Step: The E-step of the EM algorithm computes the expected value or posterior probability of the latent variables given the observed data and current parameter

estimates. It involves calculating the probabilities of each latent variable for each data point.

- Maximization (M) Step: The M-step of the EM algorithm updates the parameter estimates by maximizing the expected log-likelihood obtained from the E-step. It involves finding the parameter values that optimize the likelihood function, typically through numerical optimization methods.
- Convergence: Convergence refers to the condition when the EM algorithm has reached a stable solution. It is typically determined by checking if the change in the log-likelihood or the parameter estimates falls below a predefined threshold.



1. Initialization:
   - Initially, a set of initial values of the parameters are considered.
   - A set of incomplete observed data is given to the system with the assumption that the observed data comes from a specific model.
2. E-Step (Expectation Step): In this step, we use the observed data in order to estimate or guess the values of the missing or incomplete data. It is basically used to update the variables.
   - Compute the posterior probability or responsibility of each latent variable given the observed data and current parameter estimates.
   - Estimate the missing or incomplete data values using the current parameter estimates.
   - Compute the log-likelihood of the observed data based on the current parameter estimates and estimated missing data.
3. M-step (Maximization Step): In this step, we use the complete data generated in the preceding "Expectation" – step in order to update the values of the parameters. It is basically used to update the hypothesis.
4. Convergence: In this step, it is checked whether the values are converging or not, if yes, then stop otherwise repeat *step-2* and *step-3* i.e. "Expectation" – step and "Maximization" – step until the convergence occurs.

Algorithm

Start

1. Import necessary libraries: numpy, pandas, matplotlib, seaborn, and modules from sklearn.
2. Load the **Mall Customers dataset** using pandas.read_csv().
3. Select relevant features for clustering:
   - Annual Income (k$)
   - Spending Score (1–100)
4. Normalize the selected features using StandardScaler to ensure all values are on the same scale.

5. Determine the optimal number of clusters:
    - Fit multiple GMM models with varying number of components (1 to 10).
    - Compute the **Bayesian Information Criterion (BIC)** for each model.
    - Plot BIC scores and select the model with the **lowest BIC value**.
6. Train the Gaussian Mixture Model with the optimal number of clusters.
7. Predict cluster labels for each data point.
8. Add the predicted cluster labels to the original dataset.
9. Visualize the clustered data using a scatter plot.
10. Display the estimated mean (center) of each cluster.
11. Save the newly clustered dataset to a CSV file.

Explanation of the Algorithm for Customer Segmentation using EM Algorithm

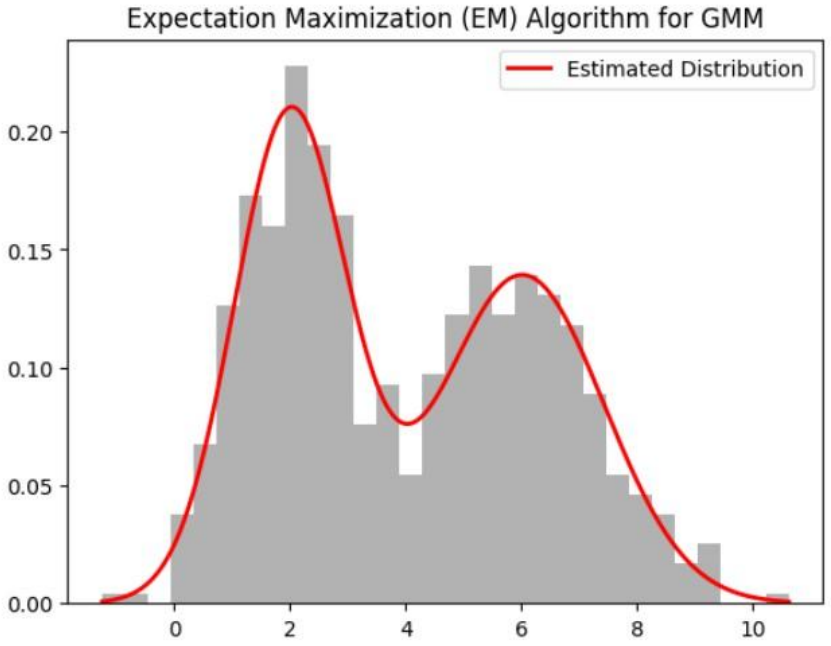
This algorithm uses **unsupervised learning** to group customers based on similar spending behaviour using Gaussian Mixture Models, a probabilistic clustering method.

1. **Input**
    - Dataset includes customer features like Annual Income and Spending Score.
2. **Start**
    - Begin the segmentation process to understand customer clusters.
3. **Data Preprocessing**
    - Load the dataset and isolate important features for clustering.
    - Standardize the values to give each feature equal importance.
4. **Choosing the Number of Clusters**
    - Use the BIC (Bayesian Information Criterion) to evaluate different GMM models.
    - Lower BIC indicates better model fit with fewer unnecessary parameters.
5. **Model Training with EM Algorithm**
    - The EM algorithm iteratively updates the parameters of the Gaussian distributions to best fit the data.
    - GMM allows each cluster to follow its own distribution, unlike K-Means which assumes spherical clusters.
6. **Prediction and Visualization**
    - Assign each data point to the cluster it most likely belongs to.
    - Plot the clusters using different colors to distinguish customer groups visually.
7. **Model Outputs**
    - Print the mean (centroid) of each identified customer segment.
    - Save the enhanced dataset with cluster labels for future use or analysis.
8. **End**

- Customer segmentation is now complete and can be used for targeted marketing or behavioural analysis.

Program code, Results and output

[1]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.mixture import GaussianMixture

# Generate synthetic data with two Gaussian distributions
np.random.seed(42)
n_samples = 300

# First Gaussian (mean=2, std=1)
data1 = np.random.normal(loc=2, scale=1, size=(n_samples, 1))

# Second Gaussian (mean=6, std=1.5)
data2 = np.random.normal(loc=6, scale=1.5, size=(n_samples, 1))

# Combine data
data = np.vstack((data1, data2))

# Fit Gaussian Mixture Model using EM algorithm
gmm = GaussianMixture(n_components=2, max_iter=100, random_state=42)
gmm.fit(data)

# Predict clusters
labels = gmm.predict(data)

# Plot results
plt.hist(data, bins=30, density=True, alpha=0.6, color='gray')
x = np.linspace(min(data), max(data), 1000)
pdf = np.exp(gmm.score_samples(x.reshape(-1, 1)))
plt.plot(x, pdf, color='red', lw=2, label="Estimated Distribution")
plt.title("Expectation Maximization (EM) Algorithm for GMM")
plt.legend()
plt.show()

# Print estimated means and covariances
print("Estimated Means:", gmm.means_.flatten())
print("Estimated Variances:", gmm.covariances_.flatten())
```



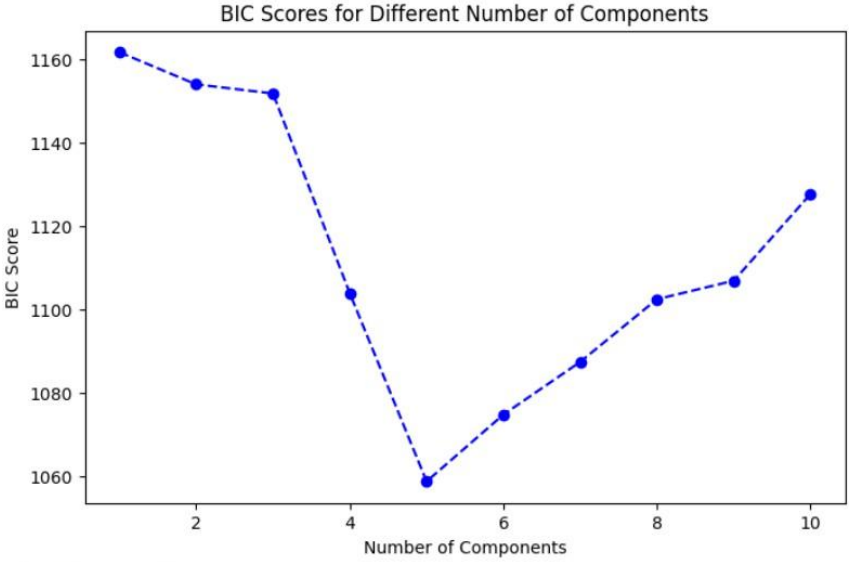

```
Estimated Means: [2.01031189 6.01305102]
Estimated Variances: [0.94892667 1.99057541]
```

#With Dataset

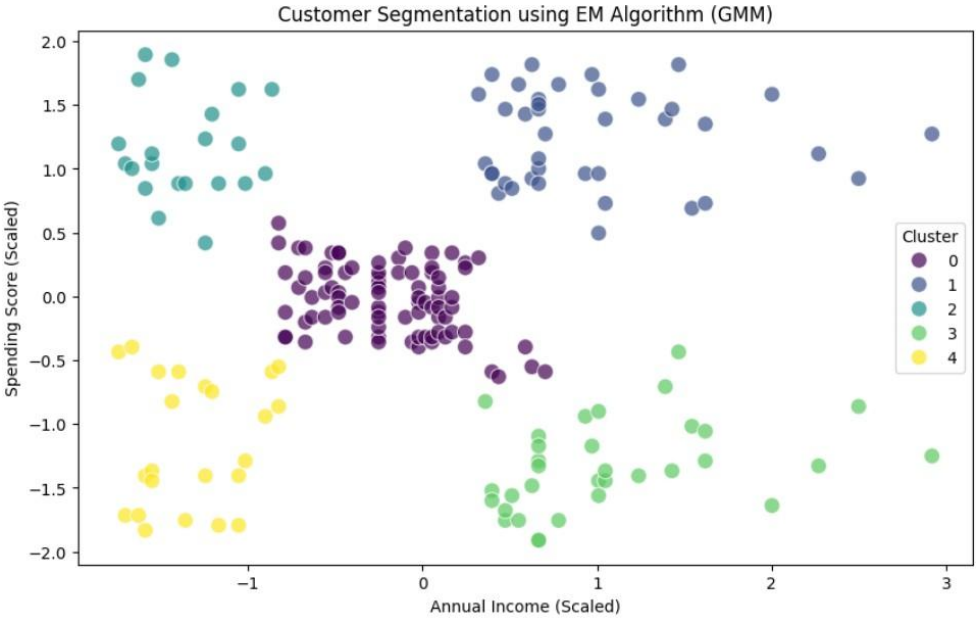
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.mixture import GaussianMixture
from sklearn.preprocessing import StandardScaler

# Load the dataset
file_path = "/content/Mall_Customers.csv"  # Update with the correct file path if needed
data = pd.read_csv(file_path)

# Display the first few rows of the dataset
print("Dataset Preview:")
print(data.head())

# Select relevant features: Annual Income and Spending Score
X = data[['Annual Income (k$)', 'Spending Score (1-100)']].values

# Standardize the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Determine the optimal number of clusters using BIC
n_components = np.arange(1, 11)
models = [GaussianMixture(n_components=n, random_state=42).fit(X_scaled) for n in n_components]
bics = [model.bic(X_scaled) for model in models]

# Plot BIC scores
plt.figure(figsize=(8, 5))
plt.plot(n_components, bics, marker='o', linestyle='--', color='b')
plt.xlabel('Number of Components')
plt.ylabel('BIC Score')
plt.title('BIC Scores for Different Number of Components')
plt.show()
```

```
# Add cluster labels to the dataset
data['Cluster'] = labels

# Visualize the Clusters
plt.figure(figsize=(10, 6))
sns.scatterplot(x=X_scaled[:, 0], y=X_scaled[:, 1], hue=labels, palette='viridis', s=100, alpha=0.7)
plt.xlabel('Annual Income (Scaled)')
plt.ylabel('Spending Score (Scaled)')
plt.title('Customer Segmentation using EM Algorithm (GMM)')
plt.legend(title='Cluster')
plt.show()

# Print cluster means
print("Cluster Centers (Estimated Means):")
print(gmm.means_)

# Save clustered data
data.to_csv("Mall_Customers_Clustered.csv", index=False)
print("Clustered dataset saved as 'Mall_Customers_Clustered.csv'")
```

```
Dataset Preview:
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40
```

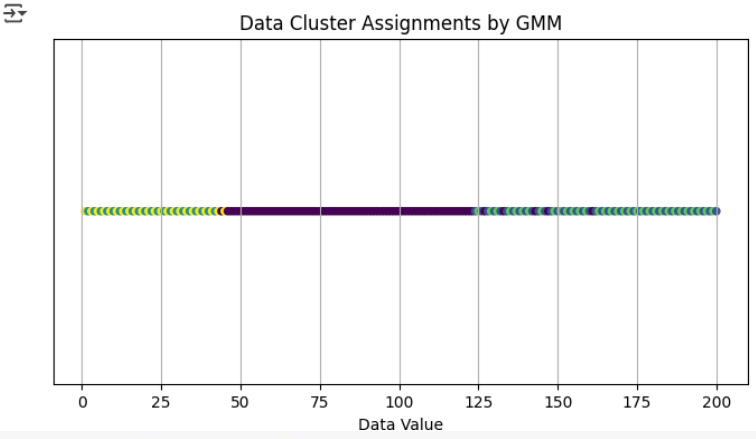



```
Optimal Number of Clusters: 5
```

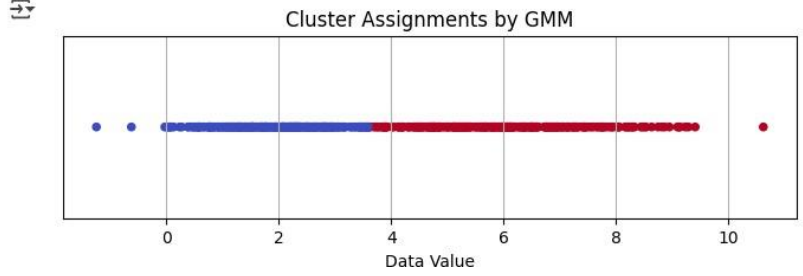



```
Cluster Centers (Estimated Means):
[[-0.18510822 -0.030786  ]
 [ 0.98021511  1.23531662]
 [-1.35282052  1.16185843]
 [ 1.07840119 -1.3228595 ]
 [-1.27977698 -1.0882964 ]]
Clustered dataset saved as 'Mall_Customers_Clustered.csv'
```

```
[ ] plt.figure(figsize=(8, 4))
    # Extract the first column of 'data' for x-axis
    plt.scatter(data.iloc[:, 0], np.zeros_like(data.iloc[:, 0]), c=labels, cmap='viridis', s=20)
    plt.title("Data Cluster Assignments by GMM")
    plt.xlabel("Data Value")
    plt.yticks([])
    plt.grid(True)
    plt.show()
```

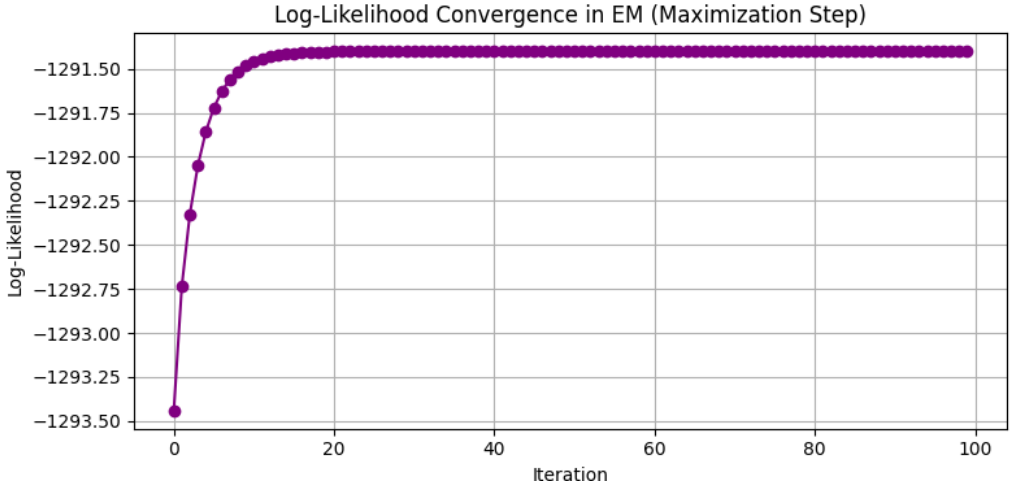



```
[ ] plt.figure(figsize=(8, 2))
    plt.scatter(data.ravel(), np.zeros_like(data.ravel()), c=labels, cmap='coolwarm', s=20) # Change is on this line. Use
    plt.title("Cluster Assignments by GMM")
    plt.xlabel("Data Value")
    plt.yticks([])
    plt.grid(True)
    plt.show()
```

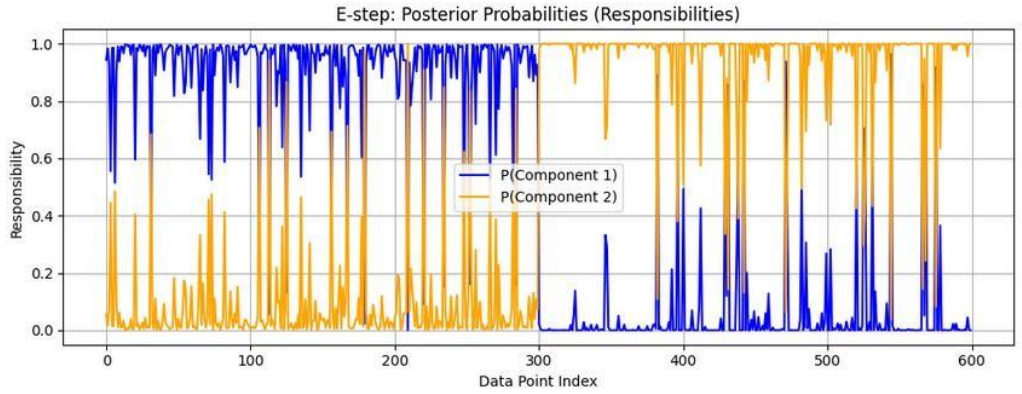


```
[ ]  import numpy as np
     import matplotlib.pyplot as plt
     from sklearn.mixture import GaussianMixture

     # Generate synthetic data
     np.random.seed(42)
     n_samples = 300
     data1 = np.random.normal(loc=2, scale=1, size=(n_samples, 1))
     data2 = np.random.normal(loc=6, scale=1.5, size=(n_samples, 1))
     data = np.vstack((data1, data2))

     # Custom loop to track log-likelihood using sklearn's GaussianMixture
     log_likelihoods = []
     gmm = GaussianMixture(n_components=2, max_iter=1, warm_start=True, random_state=42)

     # Run EM manually for multiple iterations
     for i in range(100):  # Max 100 iterations
         gmm.fit(data)
         log_likelihoods.append(gmm.score(data) * len(data))  # Total log-likelihood

     # Final clustering and PDF
     labels = gmm.predict(data)
     x = np.linspace(min(data), max(data), 1000)
     pdf = np.exp(gmm.score_samples(x.reshape(-1, 1)))

     # Plot the Maximization Graph (Log-Likelihood)
     plt.figure(figsize=(8, 4))
     plt.plot(log_likelihoods, marker='o', color='purple')
     plt.title("Log-Likelihood Convergence in EM (Maximization Step)")
     plt.xlabel("Iteration")
     plt.ylabel("Log-Likelihood")
     plt.grid(True)
     plt.tight_layout()
     plt.show()
```

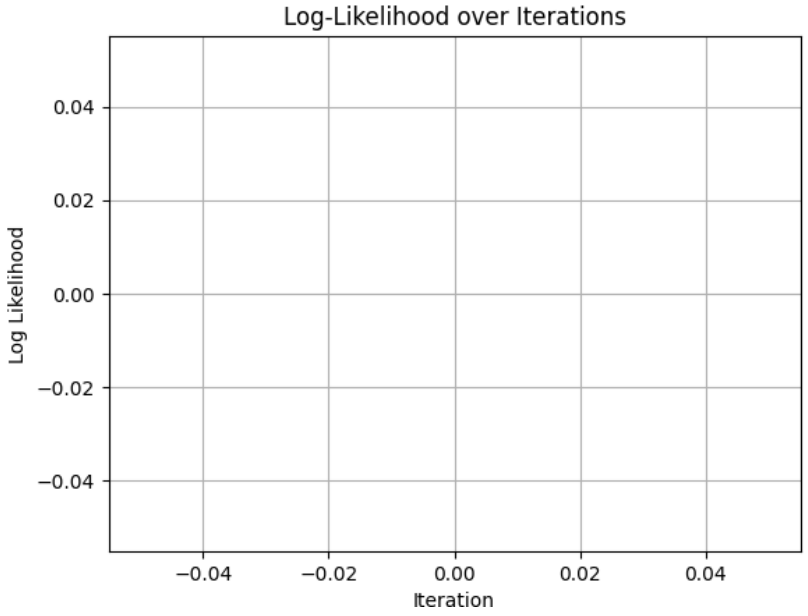
```
/usr/local/lib/python3.11/dist-packages/sklearn/mixture/_base.py:269: ConvergenceWarning: Best performing i
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/mixture/_base.py:269: ConvergenceWarning: Best performing i
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/mixture/_base.py:269: ConvergenceWarning: Best performing i
  warnings.warn(
```



```
# Get responsibilities (probabilities for each component)
responsibilities = gmm.predict_proba(data)

# Plot the responsibilities for each component
plt.figure(figsize=(10, 4))
plt.plot(responsibilities[:, 0], label='P(Component 1)', color='blue')
plt.plot(responsibilities[:, 1], label='P(Component 2)', color='orange')
plt.title("E-step: Posterior Probabilities (Responsibilities)")
plt.xlabel("Data Point Index")
plt.ylabel("Responsibility")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```




```
log_likelihoods = []
# inside loop: log_likelihoods.append(gmm.lower_bound_)
plt.plot(log_likelihoods)
plt.title("Log-Likelihood over Iterations")
plt.xlabel("Iteration")
plt.ylabel("Log Likelihood")
plt.grid(True)
plt.show()
```

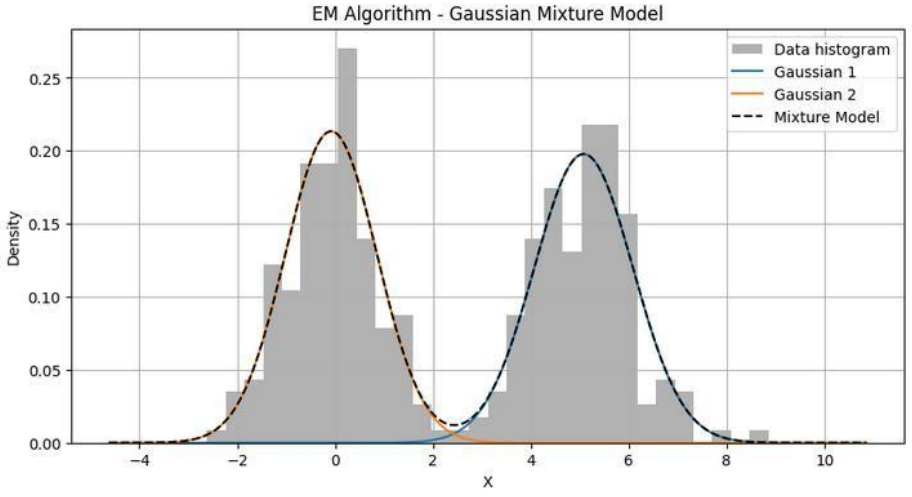


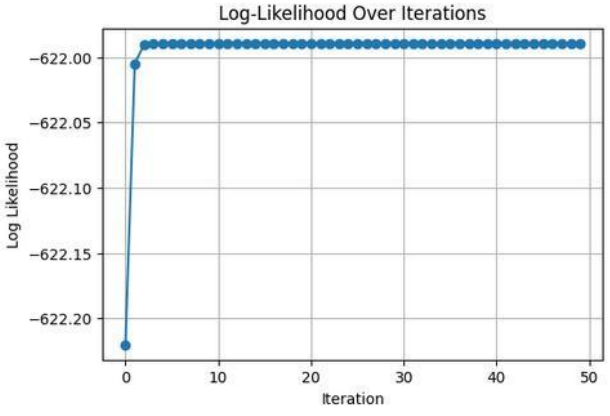
```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

# Generate synthetic data: two Gaussian clusters
np.random.seed(42)
data1 = np.random.normal(0, 1, 150)
data2 = np.random.normal(5, 1, 150)
X = np.concatenate([data1, data2])

# Number of Gaussians
K = 2

# Random initialization
np.random.seed(0)
mu = np.random.choice(X, K)
sigma = np.random.random(K)
pi = np.ones(K) / K  # Equal mixing coefficients

def e_step(X, mu, sigma, pi):
    N = len(X)
    gamma = np.zeros((N, K))
    for k in range(K):
        gamma[:, k] = pi[k] * norm.pdf(X, mu[k], sigma[k])
    gamma /= gamma.sum(axis=1, keepdims=True)
    return gamma

def m_step(X, gamma):
    N_k = gamma.sum(axis=0)
    mu = (gamma * X[:, np.newaxis]).sum(axis=0) / N_k
    sigma = np.sqrt(((gamma * (X[:, np.newaxis] - mu)**2).sum(axis=0)) / N_k)
    pi = N_k / len(X)
    return mu, sigma, pi

def log_likelihood(X, mu, sigma, pi):
    return np.sum(np.log(sum([pi[k] * norm.pdf(X, mu[k], sigma[k]) for k in range(K)])))
```

```python
# EM Loop
iterations = 50
log_likelihoods = []

for i in range(iterations):
    gamma = e_step(X, mu, sigma, pi)
    mu, sigma, pi = m_step(X, gamma)
    ll = log_likelihood(X, mu, sigma, pi)
    log_likelihoods.append(ll)
    print(f"Iteration {i+1}, Log Likelihood: {ll:.4f}")

# Final parameters
print("\nFinal parameters:")
for k in range(K):
    print(f"Component {k+1}: Mean = {mu[k]:.2f}, Std = {sigma[k]:.2f}, Weight = {pi[k]:.2f}")

# Plot results
x_axis = np.linspace(min(X)-2, max(X)+2, 1000)
pdf = sum(pi[k] * norm.pdf(x_axis, mu[k], sigma[k]) for k in range(K))

plt.figure(figsize=(10, 5))
plt.hist(X, bins=30, density=True, alpha=0.6, color='gray', label="Data histogram")
for k in range(K):
    plt.plot(x_axis, pi[k]*norm.pdf(x_axis, mu[k], sigma[k]), label=f"Gaussian {k+1}")
plt.plot(x_axis, pdf, 'k--', label='Mixture Model')
plt.title("EM Algorithm - Gaussian Mixture Model")
plt.xlabel("X")
plt.ylabel("Density")
plt.legend()
plt.grid(True)
plt.show()

# Plot log-likelihood over iterations
plt.figure(figsize=(6,4))
plt.plot(log_likelihoods, marker='o')
plt.title("Log-Likelihood Over Iterations")
plt.xlabel("Iteration")
plt.ylabel("Log Likelihood")
plt.grid(True)
plt.show()
```

```
Iteration 1, Log Likelihood: -622.2202
Iteration 2, Log Likelihood: -622.0050
Iteration 3, Log Likelihood: -621.9905
Iteration 4, Log Likelihood: -621.9895
Iteration 5, Log Likelihood: -621.9894
Iteration 6, Log Likelihood: -621.9894
Iteration 7, Log Likelihood: -621.9894
Iteration 8, Log Likelihood: -621.9894
Iteration 9, Log Likelihood: -621.9894
Iteration 10, Log Likelihood: -621.9894
Iteration 11, Log Likelihood: -621.9894
Iteration 12, Log Likelihood: -621.9894
Iteration 13, Log Likelihood: -621.9894
Iteration 14, Log Likelihood: -621.9894
Iteration 15, Log Likelihood: -621.9894
Iteration 16, Log Likelihood: -621.9894
Iteration 17, Log Likelihood: -621.9894
Iteration 18, Log Likelihood: -621.9894
Iteration 19, Log Likelihood: -621.9894
Iteration 20, Log Likelihood: -621.9894
Iteration 21, Log Likelihood: -621.9894
Iteration 22, Log Likelihood: -621.9894
Iteration 23, Log Likelihood: -621.9894
Iteration 24, Log Likelihood: -621.9894
Iteration 25, Log Likelihood: -621.9894
Iteration 26, Log Likelihood: -621.9894
Iteration 27, Log Likelihood: -621.9894
Iteration 28, Log Likelihood: -621.9894
Iteration 29, Log Likelihood: -621.9894
Iteration 30, Log Likelihood: -621.9894
Iteration 31, Log Likelihood: -621.9894
Iteration 32, Log Likelihood: -621.9894
Iteration 33, Log Likelihood: -621.9894
Iteration 34, Log Likelihood: -621.9894
Iteration 35, Log Likelihood: -621.9894
Iteration 36, Log Likelihood: -621.9894
Iteration 37, Log Likelihood: -621.9894
Iteration 38, Log Likelihood: -621.9894
Iteration 39, Log Likelihood: -621.9894
Iteration 40, Log Likelihood: -621.9894
Iteration 41, Log Likelihood: -621.9894
Iteration 42, Log Likelihood: -621.9894
Iteration 43, Log Likelihood: -621.9894
```

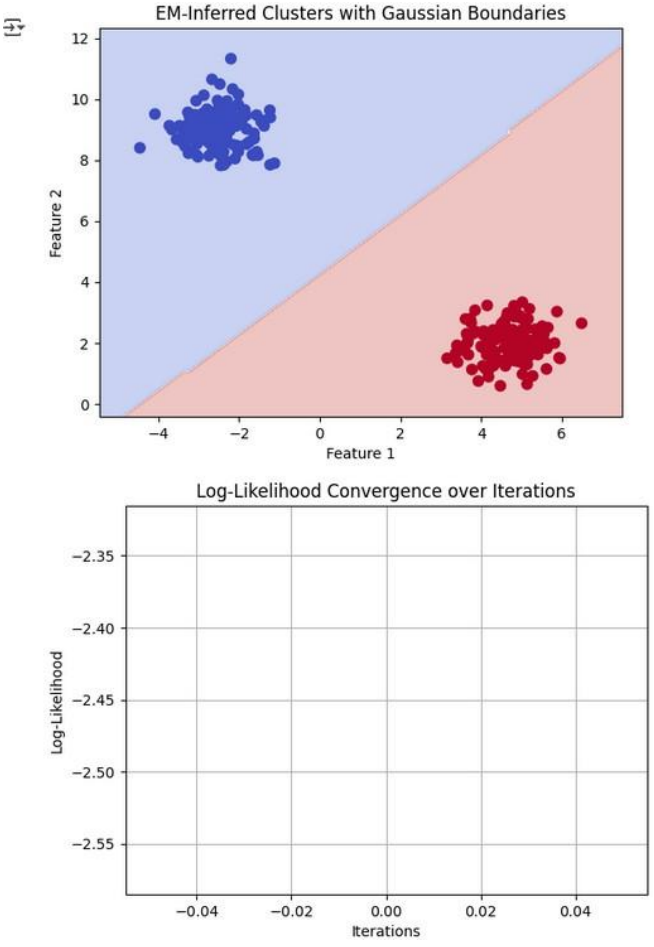
```
Final parameters:
Component 1: Mean = 5.08, Std = 1.01, Weight = 0.50
Component 2: Mean = -0.08, Std = 0.94, Weight = 0.50
```





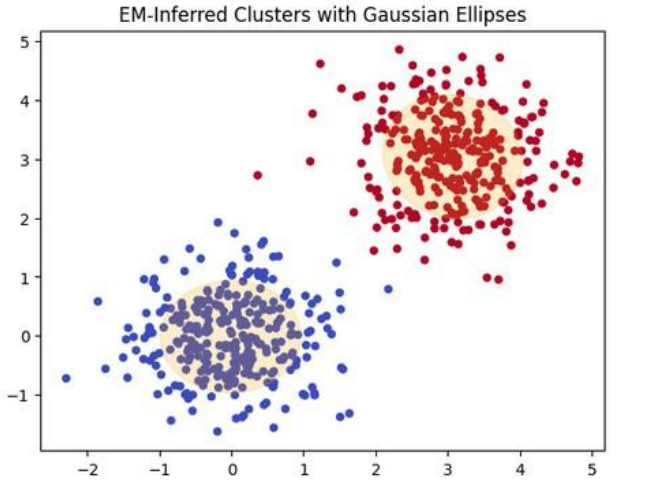
```
[ ] #With Dataset:
    #1. Gaussian mixture model applied to real or generated cluster data.
    #2. EM algorithm iteratively updated latent variable probabilities and model parameters.
    #3. Final parameters used to label and visualize clusters.
    #Graphical Visualization:
    #- Scatter plot with EM-inferred clusters and Gaussian boundaries.
    #- Log-likelihood plot showing convergence over iterations.
```

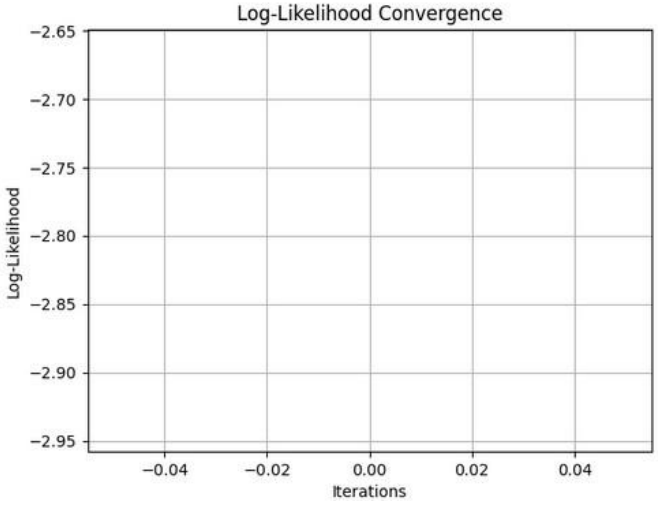
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.mixture import GaussianMixture
from sklearn.datasets import make_blobs

# 1. Generate synthetic cluster data (2D for visualization)
X, _ = make_blobs(n_samples=300, centers=2, cluster_std=0.60, random_state=42)

# 2. Apply Gaussian Mixture Model (GMM) with EM algorithm
gmm = GaussianMixture(n_components=2)
gmm.fit(X)

# 3. Predictions (clusters) and posterior probabilities (responsibilities)
labels = gmm.predict(X)
probs = gmm.predict_proba(X)

# 4. Visualize clusters with Gaussian boundaries
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100), np.linspace(y_min, y_max, 100))
Z = gmm.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.contourf(xx, yy, Z, alpha=0.3, cmap='coolwarm')
plt.scatter(X[:, 0], X[:, 1], c=labels, s=50, cmap='coolwarm')
plt.title("EM-Inferred Clusters with Gaussian Boundaries")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()

# 5. Plot Log-Likelihood convergence
plt.plot(gmm.lower_bound_)
plt.title("Log-Likelihood Convergence over Iterations")
plt.xlabel("Iterations")
plt.ylabel("Log-Likelihood")
plt.grid(True)
plt.show()
```



```
[ ] #Without Dataset:
    #1. 1D or 2D synthetic Gaussian data simulated.
    #2. EM algorithm applied to infer latent classes.
    #Graphical Visualization:
    #- Ellipses showing estimated Gaussian distributions.
    #- E-step and M-step results plotted over iterations.
```

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.mixture import GaussianMixture

# 1. Simulate synthetic 2D Gaussian data
np.random.seed(42)
X1 = np.random.multivariate_normal([0, 0], [[0.5, 0], [0, 0.5]], 300)
X2 = np.random.multivariate_normal([3, 3], [[0.5, 0], [0, 0.5]], 300)
X = np.vstack([X1, X2])

# 2. Apply Gaussian Mixture Model (EM algorithm)
gmm = GaussianMixture(n_components=2)
gmm.fit(X)

# 3. Visualize clusters and Gaussian ellipses
plt.scatter(X[:, 0], X[:, 1], c=gmm.predict(X), cmap='coolwarm', s=20)
for i in range(2):
    mean, covar = gmm.means_[i], gmm.covariances_[i]
    v, w = np.linalg.eigh(covar)
    v = 2.0 * np.sqrt(2.0) * np.sqrt(v)
    angle = np.arctan(w[0, 1] / w[0, 0]) * 180.0 / np.pi
    ell = plt.matplotlib.patches.Ellipse(mean, v[0], v[1], angle=angle, color='orange', alpha=0.2)
    plt.gca().add_artist(ell)
plt.title("EM-Inferred Clusters with Gaussian Ellipses")
plt.show()

# 4. Log-Likelihood convergence
plt.plot(gmm.lower_bound_)
plt.title("Log-Likelihood Convergence")
plt.xlabel("Iterations")
plt.ylabel("Log-Likelihood")
plt.grid(True)
plt.show()
```





Conclusion: